# vmd3_sar_processing_pipeline

August 14, 2026

## 1 V-MD3 Rail-SAR Data Validation and Processing Pipeline

**Four-RX SAR imaging of Runs C and D**
Prepared for technical review and collaboration · 14 August 2026

This notebook presents the data-validation and image-formation pipeline used for the V-MD3 rail-SAR proof-of-concept measurements. It is intended to give collaborators a clear overview of the acquired data, the order of the processing operations, the purpose of each stage, and how the stages improve the final SAR image.

The pipeline uses the following original binary acquisitions:

- **Run B:** matched no-target background scan;
- **Run C:** sphere near the center of the scene at a laser-measured 0.84 m slant range;
- **Run D:** sphere at the same laser-measured 0.84 m slant range, displaced to the **right** of the Run C target.

Run A is not included because it belongs to the conventional-radar analysis rather than the rail-SAR image-formation path. Parameter sweeps, debugging cells, and development-only diagnostics are also omitted so the notebook remains focused on the validated processing sequence. The same processing configuration is applied to Runs C and D; Run D provides a transfer check for the calibrated pipeline and the positive-cross-range convention.

### 1.1 Overview

The V-MD3 records complex FMCW ADC data from four physical RX channels at 17 radar positions along a 10 cm mechanical rail. At each position, 20 repeated frames are collected. The pipeline preserves all four channels through range processing, zero-Doppler extraction, coherent frame averaging, matched Run B subtraction, per-channel range calibration, and near-field backprojection. The four independently focused images are then combined using equal or quality-weighted power fusion. An experimental coherent-RX result is included as a separate comparison.

**Processing baseline.** The baseline product includes fractional background alignment, Run B subtraction, per-RX range calibration, independent near-field backprojection, and equal-power fusion. It does not include the smooth aperture-phase correction, RX quality weighting, or coherent inter-RX combination. This defined baseline is retained at the end of the notebook so the effect of the additional calibration and fusion stages can be compared directly.

The notebook reports three focused SAR products for each run: the defined equal-power baseline, the recommended calibrated quality-weighted power image, and an explicitly experimental coherent-RX comparison. With the image convention used here, positive cross-range is displayed

to the right; therefore, the Run D peak is expected to lie at a more positive cross-range coordinate than the Run C peak.

## 1.2 Processing flowgraph

```
flowchart TD
    A["Run B, C, and D binary files"] --> B["Parse records and validate RADC/DONE streams"]
    B --> C["Build complex RADC cubes"]
    C --> D["Preserve all four RX channels"]
    D --> E["Per-RX Hamming window + 1024-point range FFT"]
    E --> F["Hann chirp window + zero-Doppler extraction"]
    F --> G["Coherent average of 20 frames per position"]
    G --> H["Fractional range-shift alignment + Run B subtraction"]
    H --> I["Fractional-bin per-RX range calibration from Run C"]
    I --> J["Reference baseline: per-RX backprojection"]
    J --> K["Smooth Run C aperture-phase calibration"]
    K --> L["Calibrated per-RX backprojection"]
    L --> M["Run C-derived RX quality weights"]
    M --> N["Weighted power fusion"]
    N --> O["Experimental coherent RX fusion"]
    O --> P["Run D transfer validation + image metrics"]
```

### 1.2.1 What each stage contributes

1. **Parsing and validation** preserve the complex V-MD3 samples and verify file counts, payload sizes, frame counts, and DONE-counter continuity.
2. **Range/Doppler processing** converts fast time into range while retaining the stationary return used in this static-target experiment.
3. **Coherent frame averaging** improves repeatability without discarding phase.
4. **Fractional-shift matched-background subtraction** tests small sub-bin translations of Run B, then estimates a complex scale factor at every rail position and RX. This corrects small range-grid drift before subtraction instead of assuming that both runs align perfectly at the same bin coordinates.
5. **Fractional-bin per-RX range calibration** fits the Run C sphere peak between FFT-grid samples and maps that measured location to the laser-measured 0.84 m range. The four coefficients apply unchanged to Run D and compatible future data.
6. **Reference baseline per-RX backprojection** forms one SAR image per RX using the calibrated range mapping and the original measured aperture phase. Equal-power fusion of these images defines the notebook's controlled baseline for evaluating later improvements.
7. **Smooth aperture-phase calibration** uses the Run C target to estimate only a low-order phase trend across the rail. The frozen correction is transferred to Run D to test whether it represents system phase error rather than Run C-specific noise.
8. **Quality-weighted power fusion** gives cleaner, more coherent RX channels greater influence while limiting how strongly any one channel can dominate.
9. **Experimental coherent fusion** estimates constant inter-RX phase offsets from Run C and tests a coherent complex sum on Run D. It is reported alongside—not substituted for—the more conservative power-fused result.
10. **Transfer validation and metrics** compare peak location, range error, widths, sidelobes,

and focusing coherence before accepting an apparent improvement.

### 1.3 Pipeline walkthrough

The processing begins by reading the V-MD3 binary records for Runs B, C, and D. Each RADC payload is decoded into a complex cube whose axes are fast-time sample, chirp, and physical RX channel. The four RX channels remain separate throughout range processing; they are not beamformed or averaged together at this stage. Run B contains the matched empty-scene measurement, while Runs C and D contain the sphere.

Next, each channel is processed independently. A Hamming window is applied across the 128 fast-time samples before a 1024-point range FFT. The 1024-point grid is an interpolated representation of the spectrum measured by the original 128 samples. A Hann window is then applied across the 64 chirps, and the complex zero-Doppler response is retained because the rail, room, and sphere are stationary within each frame. The 20 repeated frames at every rail position are coherently averaged, preserving the phase needed for SAR.

Static clutter is removed before image formation. For every rail position and RX channel, the pipeline searches a small set of fractional range-grid shifts for Run B. At each candidate shift it also estimates the best complex amplitude/phase coefficient, using range bins that exclude the sphere. The shift and coefficient that produce the highest background correlation are retained before subtraction. This compensates small run-to-run range registration drift that a single complex coefficient cannot remove.

Run C then supplies the current V-MD3 range calibration. The sphere was laser-measured at a slant range of 0.84 m but appeared near original range bin 22, whose nominal value was approximately 1.03 m. Instead of rounding the response to one 1024-point FFT sample, the pipeline fits the local peak and estimates a fractional bin coordinate. A separate additive range correction is calculated for each RX. These coefficients belong to the waveform configuration—not to Run D—and are frozen before Run D is evaluated.

A baseline SAR image is first formed independently for each RX. For a candidate image pixel, backprojection predicts the distance from all 17 phase-center positions, converts each distance to a calibrated fractional range bin, interpolates the complex data, removes the predicted rail-dependent phase, and coherently sums the aperture. The Run C baseline peak then provides a calibration location for examining the remaining phase error across the rail.

At the Run C peak, the pipeline compares the measured aperture phase with the geometric prediction and fits only a smooth, low-order phase trend. That correction is applied to every range bin and then frozen for Run D. The target runs are backprojected again, producing phase-calibrated per-RX images. Transfer to Run D is essential: a correction that cleans Run C but degrades or misplaces Run D is treated as overfitting rather than an improvement.

The calibrated channel images are combined in two ways. The primary result is **quality-weighted power fusion**: each magnitude-squared image is multiplied by a Run C-derived channel-quality weight and added pixel by pixel. The weights use target-to-background ratio, frame coherence, and background correlation, with clipping that prevents one RX from dominating. An experimental coherent image is also formed after estimating constant inter-RX phase offsets at the Run C calibration point. Both results are displayed and measured; coherent fusion remains experimental until it transfers successfully to Run D.

The final tables compare baseline equal-power fusion, calibrated weighted-power fusion, and calibrated coherent fusion using target location, laser-range error, response widths, focusing coherence, and sidelobe metrics. This keeps the decision evidence-based rather than selecting whichever image merely looks cleanest.

### 1.4 V-MD3 data model and fixed acquisition geometry

| Data product | Shape | Axis meaning |
|---|---|---|
| One decoded RADC frame | `(128, 64, 4)` | fast-time sample, chirp, RX channel |
| One complete rail run | `(17, 20, 128, 64, 4)` | rail position, frame, sample, chirp, RX |
| Four-RX oversampled zero-Doppler stack | `(17, 20, 1024, 4)` | rail position, frame, oversampled range bin, RX |
| Coherently averaged profiles | `(17, 1024, 4)` | rail position, oversampled range bin, RX |
| Background-subtracted aperture data | `(17, 1024, 4)` | rail position, oversampled range bin, RX |
| Four independently focused images | `(4, 176, 201)` | RX, downrange pixel, cross-range pixel |
| Four-RX fused image power | `(176, 201)` | downrange pixel, cross-range pixel |

The binary files contain interleaved `RADC`, `RFFT`, and `DONE` records. This pipeline decodes `RADC` and `DONE`; the stored FPGA `RFFT` stream is indexed and validated but is not used for SAR formation.

| Parameter | Value used here |
|---|---|
| Approximate carrier frequency | 61.0 GHz |
| RX element spacing | 2.464 mm |
| Physical receive channels | RX 0–3, all retained |
| Mechanical aperture | 10 cm |
| Rail positions | 17, centered at index 8 |
| Rail increment | 6.25 mm (50,000 encoder counts) |
| Repeated frames per position | 20 |
| Nominal uncalibrated range span | 6 m over 128 measured bins |
| Range processing | Hamming window, 1024-point FFT |
| Slow-time processing | Hann window, zero Doppler |
| Aperture weighting | Hamming |
| Measured propagation-phase sign | +1 |

| Enhanced-processing setting | Fixed value |
|---|---|
| Background range-shift search | $\pm 2$ oversampled bins, 81 trials |
| Range-reference peak estimator | three-sample quadratic fit in log power |
| Aperture-phase model | degree-2 polynomial versus rail position |

| Enhanced-processing setting | Fixed value |
|---|---|
| Maximum RX-weight spread | 4:1 before normalization |
| Calibration source | Run C at 0.84 m measured slant range |
| Transfer-validation run | Run D; no parameter refitting |

Zero-padding from 128 to 1024 points makes the complex range spectrum more densely sampled for interpolation. It does **not** add bandwidth or improve the physical range resolution.

### 1.5 1. Configuration

The repository and data paths below are unchanged from the source notebook. Processing parameters are grouped in one cell so collaborators can audit the assumptions without searching through the notebook.

[1]:
```
from pathlib import Path
import re
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

# Original V-MD3 dataset locations
REPO_ROOT = Path(r"D:\Research\repos\vmd3_projects")
DATA_ROOT = REPO_ROOT / "data"
RUN_DIRS = {
    run: DATA_ROOT / f"20260709_run_{run.lower()}" / "bins"
    for run in ("B", "C", "D")
}

# Acquisition geometry
SAR_POSITION_INDICES = np.arange(17)
CENTER_POSITION_INDEX = 8
GALIL_ENCODER_COUNTS = np.arange(0, 800_000 + 50_000, 50_000)
SAR_APERTURE_LENGTH_M = 0.10

# V-MD3 array and waveform assumptions validated in the source notebook
CARRIER_FREQUENCY_HZ = 61.0e9
RX_ELEMENT_SPACING_M = 2.464e-3
RX_CHANNELS = np.arange(4)
MEASURED_PHASE_SIGN = +1
NOMINAL_MAXIMUM_RANGE_M = 6.0

# Locked signal-processing configuration
RANGE_WINDOW = "hamming"
DOPPLER_WINDOW = "hann"
```

```
RANGE_FFT_SIZE = 1024
APERTURE_WINDOW = "hamming"
BACKGROUND_MINIMUM_REFERENCE_LEVEL_DB = -35.0
BACKGROUND_SHIFT_SEARCH_BINS = 2.0
BACKGROUND_SHIFT_TRIALS = 81

# Laser reference used to calibrate the V-MD3 range mapping.
# The reference was recorded in Run C, but the fitted coefficients
# apply to every dataset acquired with the same waveform settings.
CALIBRATION_SOURCE_RUN = "C"
LASER_REFERENCE_SLANT_RANGE_M = 0.84
EXPECTED_TARGET_SLANT_RANGE_M = {"C": 0.84, "D": 0.84}
APERTURE_PHASE_POLYNOMIAL_DEGREE = 2
RX_WEIGHT_MAX_TO_MIN_RATIO = 4.0
TARGET_SEARCH_ORIGINAL_BINS = (19.0, 26.0)
BACKGROUND_REFERENCE_ORIGINAL_BINS = (1.0, 51.0)
BACKGROUND_EXCLUSION_ORIGINAL_BINS = (15.0, 27.0)

# Image grid; positive cross-range is displayed to the right
IMAGE_CROSS_RANGE_M = np.linspace(-0.10, 0.10, 201)
IMAGE_DOWNRANGE_M = np.linspace(0.70, 1.05, 176)
IMAGE_DISPLAY_FLOOR_DB = -25.0
METRIC_SEARCH_CROSS_RANGE_M = 0.15
METRIC_SEARCH_DOWNRANGE_M = (0.70, 1.00)

assert np.array_equal(RX_CHANNELS, np.arange(4))
assert len(SAR_POSITION_INDICES) == len(GALIL_ENCODER_COUNTS) == 17

print("Repository:", REPO_ROOT)
for run, folder in RUN_DIRS.items():
    print(f"Run {run}: {folder} | exists = {folder.exists()}")
```

```
Repository: D:\Research\repos\vmd3_projects
Run B: D:\Research\repos\vmd3_projects\data\20260709_run_b\bins | exists = True
Run C: D:\Research\repos\vmd3_projects\data\20260709_run_c\bins | exists = True
Run D: D:\Research\repos\vmd3_projects\data\20260709_run_d\bins | exists = True
```

### 1.6 2. Binary parsing and manifests

Each stored record begins with a four-byte stream code and a four-byte little-endian payload length. The functions below scan record boundaries before decoding payloads, so malformed headers, unexpected payload sizes, missing frames, or discontinuous DONE counters stop the pipeline early.

[2]:
```
_POSITION_PATTERN = re.compile(
    r"^scan_(?:pos)?(?P<position>\d+)cm\.bin$",
    flags=re.IGNORECASE,
)
```

```
FRAME_HEADER_BYTES = 8
STREAM_CODE_TO_NAME = {b"RADC": "radc", b"RFFT": "rfft", b"DONE": "done"}
EXPECTED_PAYLOAD_BYTES = {"radc": 131_072, "rfft": 131_072, "done": 4}
RADC_FRAME_SHAPE = (128, 64, 4)

def parse_sar_position_index(filename: str) -> int:
    """
    Extract the acquisition position index from a Run B, C, or D filename.

    Examples
    --------
    scan_0cm.bin     -> 0
    scan_pos8cm.bin  -> 8
    scan_pos16cm.bin -> 16

    The extracted number is treated as a position index, not as a
    trustworthy physical distance in centimeters.
    """
    match = _POSITION_PATTERN.fullmatch(filename)

    if match is None:
        raise ValueError(
            f"Could not determine position index from filename: {filename}"
        )

    return int(match.group("position"))


def scan_vmd3_bin(path: Path) -> pd.DataFrame:
    """
    Scan one acquisition .bin file and return one row per stored frame.

    Stored frame format
    -------------------
    bytes 0:4   Stream code: RADC, RFFT, or DONE
    bytes 4:8   Payload length, little-endian unsigned integer
    bytes 8:    Payload
    """
    path = Path(path)
    file_size = path.stat().st_size

    rows = []
    stream_counts = {
        stream_name: 0
        for stream_name in STREAM_CODE_TO_NAME.values()
    }
```

```
    offset = 0
    record_index = 0

    with path.open("rb") as file:
        while offset < file_size:
            file.seek(offset)
            header = file.read(FRAME_HEADER_BYTES)

            if len(header) != FRAME_HEADER_BYTES:
                raise EOFError(
                    f"{path.name}: incomplete frame header "
                    f"at byte offset {offset:,}."
                )

            stream_code = header[:4]

            if stream_code not in STREAM_CODE_TO_NAME:
                raise ValueError(
                    f"{path.name}: unknown stream code "
                    f"{stream_code!r} at byte offset {offset:,}."
                )

            stream = STREAM_CODE_TO_NAME[stream_code]

            payload_length = int.from_bytes(
                header[4:8],
                byteorder="little",
                signed=False,
            )

            expected_length = EXPECTED_PAYLOAD_BYTES[stream]

            if payload_length != expected_length:
                raise ValueError(
                    f"{path.name}: {stream.upper()} record "
                    f"{record_index} has payload length "
                    f"{payload_length:,}; expected "
                    f"{expected_length:,}."
                )

            payload_offset = offset + FRAME_HEADER_BYTES
            next_offset = payload_offset + payload_length

            if next_offset > file_size:
                raise EOFError(
                    f"{path.name}: record {record_index} extends "
                    "beyond the end of the file."
```

```
                )

            stream_frame_index = stream_counts[stream]
            stream_counts[stream] += 1

            rows.append(
                {
                    "record_index": record_index,
                    "stream": stream,
                    "stream_frame_index": stream_frame_index,
                    "header_offset": offset,
                    "payload_offset": payload_offset,
                    "payload_length": payload_length,
                    "frame_length": (
                        FRAME_HEADER_BYTES + payload_length
                    ),
                }
            )

            offset = next_offset
            record_index += 1

    if offset != file_size:
        raise ValueError(
            f"{path.name}: parser stopped at byte {offset:,}, "
            f"but the file size is {file_size:,}."
        )

    return pd.DataFrame(rows)


def read_record_payload(
    path: Path,
    payload_offset: int,
    payload_length: int,
) -> bytes:
    """Read one payload from a parsed V-MD3 record."""
    path = Path(path)

    with path.open("rb") as file:
        file.seek(int(payload_offset))
        payload = file.read(int(payload_length))

    if len(payload) != int(payload_length):
        raise EOFError(
            f"{path.name}: requested {payload_length:,} bytes "
            f"at offset {payload_offset:,}, but read "
```

```
            f"{len(payload):,} bytes."
        )

    return payload


def decode_radc_2d(payload: bytes) -> np.ndarray:
    """
    Decode one mode-0 RADC payload.

    Returns
    -------
    cube
        Shape: (sample, chirp, channel) = (128, 64, 4)
    """
    expected_bytes = EXPECTED_PAYLOAD_BYTES["radc"]

    if len(payload) != expected_bytes:
        raise ValueError(
            f"Expected {expected_bytes:,} RADC bytes, "
            f"received {len(payload):,}."
        )

    # Stored as:
    # chirp, channel, interleaved [Q0, I0, Q1, I1, ...]
    raw = np.frombuffer(
        payload,
        dtype="<i2",
    ).reshape(64, 4, 256)

    q = raw[:, :, 0::2].astype(np.float64)
    i = raw[:, :, 1::2].astype(np.float64)

    # Convert from (chirp, channel, sample)
    # to (sample, chirp, channel).
    cube = np.transpose(
        i + 1j * q,
        (2, 0, 1),
    )

    if cube.shape != RADC_FRAME_SHAPE:
        raise ValueError(
            f"Decoded RADC shape is {cube.shape}; "
            f"expected {RADC_FRAME_SHAPE}."
        )

    return cube
```

```python
def decode_done(payload: bytes) -> int:
    """
    Decode one DONE payload as a little-endian unsigned integer.

    The values appear to behave as radar-side frame counters.
    """
    expected_bytes = EXPECTED_PAYLOAD_BYTES["done"]

    if len(payload) != expected_bytes:
        raise ValueError(
            f"Expected {expected_bytes} DONE bytes, "
            f"received {len(payload)}."
        )

    return int.from_bytes(
        payload,
        byteorder="little",
        signed=False,
    )


def _select_stream_records(
    record_manifest: pd.DataFrame,
    run: str,
    stream: str,
    position_index: int | None = None,
) -> pd.DataFrame:
    """Select and order records for one run, stream, and position."""
    run = run.upper()
    stream = stream.lower()

    if run not in {"A", "B", "C", "D"}:
        raise ValueError(
            f"Unknown run {run!r}. Expected A, B, C, or D."
        )

    if stream not in {"radc", "rfft", "done"}:
        raise ValueError(
            f"Unknown stream {stream!r}."
        )

    records = record_manifest.loc[
        (record_manifest["run"] == run)
        & (record_manifest["stream"] == stream)
    ].copy()
```

```
    if run == "A":
        if position_index is not None:
            raise ValueError(
                "Run A is fixed-position and does not use "
                "a SAR position index."
            )
    else:
        if position_index is None:
            raise ValueError(
                f"position_index is required for Run {run}."
            )

        position_index = int(position_index)

        if position_index not in SAR_POSITION_INDICES:
            raise ValueError(
                f"Position index {position_index} is outside "
                "the expected range 0 through 16."
            )

        records = records.loc[
            records["position_index"] == position_index
        ]

    records = records.sort_values("stream_frame_index")

    if records.empty:
        raise ValueError(
            f"No {stream.upper()} records found for Run {run}, "
            f"position {position_index}."
        )

    return records


def load_radc_stack(
    record_manifest: pd.DataFrame,
    run: str,
    position_index: int | None = None,
) -> np.ndarray:
    """
    Load RADC data with shape:

        (frame, sample, chirp, channel)
    """
    records = _select_stream_records(
```

```
        record_manifest,
        run,
        "radc",
        position_index,
    )

    cubes = []

    for record in records.itertuples(index=False):
        payload = read_record_payload(
            record.path,
            record.payload_offset,
            record.payload_length,
        )

        cubes.append(decode_radc_2d(payload))

    return np.stack(cubes, axis=0)


def load_done_values(
    record_manifest: pd.DataFrame,
    run: str,
    position_index: int | None = None,
) -> np.ndarray:
    """Load the DONE values for one run or SAR position."""
    records = _select_stream_records(
        record_manifest,
        run,
        "done",
        position_index,
    )

    values = []

    for record in records.itertuples(index=False):
        payload = read_record_payload(
            record.path,
            record.payload_offset,
            record.payload_length,
        )

        values.append(decode_done(payload))

    return np.asarray(values, dtype=np.uint32)
```

```python
def validate_record_manifest(
    record_manifest: pd.DataFrame,
) -> None:
    """
    Validate the stored records in every acquisition file.

    The acquisition code saves valid RADC, RFFT, and DONE frames in
    arrival order. Therefore, this function does not require repeating
    RADC-RFFT-DONE triplets.
    """
    expected_frames_per_file = {
        "A": 5,
        "B": 20,
        "C": 20,
        "D": 20,
    }

    expected_streams = ("radc", "rfft", "done")

    for (run, filename), group in record_manifest.groupby(
        ["run", "filename"],
        sort=False,
    ):
        group = group.sort_values("record_index").reset_index(drop=True)

        expected_count = expected_frames_per_file[run]
        expected_total_records = expected_count * len(expected_streams)

        # Check total number of stored records.
        if len(group) != expected_total_records:
            raise ValueError(
                f"Run {run}, {filename}: expected "
                f"{expected_total_records} total records, "
                f"found {len(group)}."
            )

        # Check that record indices are sequential.
        actual_record_indices = group["record_index"].astype(int).to_numpy()
        expected_record_indices = np.arange(expected_total_records)

        if not np.array_equal(
            actual_record_indices,
            expected_record_indices,
        ):
            raise ValueError(
                f"Run {run}, {filename}: record indices are "
                "not consecutive."
```

```
        )

    # Validate each stream independently.
    for stream in expected_streams:
        stream_group = (
            group.loc[group["stream"] == stream]
            .sort_values("stream_frame_index")
        )

        if len(stream_group) != expected_count:
            raise ValueError(
                f"Run {run}, {filename}: expected "
                f"{expected_count} {stream.upper()} records, "
                f"found {len(stream_group)}."
            )

        actual_stream_indices = (
            stream_group["stream_frame_index"]
            .astype(int)
            .to_numpy()
        )

        expected_stream_indices = np.arange(expected_count)

        if not np.array_equal(
            actual_stream_indices,
            expected_stream_indices,
        ):
            raise ValueError(
                f"Run {run}, {filename}: "
                f"{stream.upper()} frame indices are "
                "not consecutive."
            )

        expected_payload_length = EXPECTED_PAYLOAD_BYTES[stream]

        if not np.all(
            stream_group["payload_length"].to_numpy()
            == expected_payload_length
        ):
            raise ValueError(
                f"Run {run}, {filename}: one or more "
                f"{stream.upper()} payload lengths are incorrect."
            )

    # DONE should behave as a consecutive radar-side counter
    # within an acquisition file.
```

```
        done_values = (
            group.loc[group["stream"] == "done", "done_value"]
            .dropna()
            .astype(np.int64)
            .to_numpy()
        )

        if len(done_values) != expected_count:
            raise ValueError(
                f"Run {run}, {filename}: expected "
                f"{expected_count} decoded DONE values, "
                f"found {len(done_values)}."
            )

        if (
            len(done_values) > 1
            and not np.all(np.diff(done_values) == 1)
        ):
            raise ValueError(
                f"Run {run}, {filename}: DONE values are not "
                f"consecutive: {done_values.tolist()}"
            )

    print("Record-manifest validation passed for all files.")
    print(
        "Note: RADC, RFFT, and DONE arrival order was not "
        "required to form repeating triplets."
    )
```

[3]:
```
# Discover only the three rail runs used in this notebook.
BIN_FILES = {
    run: sorted(folder.glob("*.bin"))
    for run, folder in RUN_DIRS.items()
}

expected_file_counts = {"B": 17, "C": 17, "D": 17}
actual_file_counts = {run: len(files) for run, files in BIN_FILES.items()}
if actual_file_counts != expected_file_counts:
    raise ValueError(
        f"Expected 17 binary files for each of Runs B, C, and D; "
        f"found {actual_file_counts}. Check REPO_ROOT and the data folders."
    )

file_rows = []
for run, files in BIN_FILES.items():
    for path in files:
        position_index = parse_sar_position_index(path.name)
```

```
        if position_index not in SAR_POSITION_INDICES:
            raise ValueError(f"Unexpected position index in {path.name}.")
        file_rows.append(
            {
                "run": run,
                "acquisition_type": "sar_scan",
                "position_index": position_index,
                "encoder_count": int(GALIL_ENCODER_COUNTS[position_index]),
                "filename": path.name,
                "file_size_bytes": path.stat().st_size,
                "path": path,
            }
        )

FILE_MANIFEST = (
    pd.DataFrame(file_rows)
    .sort_values(["run", "position_index"])
    .reset_index(drop=True)
)
FILE_MANIFEST["position_index"] = FILE_MANIFEST["position_index"].
 ↪astype("Int64")
FILE_MANIFEST["encoder_count"] = FILE_MANIFEST["encoder_count"].astype("Int64")

expected_positions = set(SAR_POSITION_INDICES.tolist())
for run in ("B", "C", "D"):
    run_rows = FILE_MANIFEST.loc[FILE_MANIFEST["run"] == run]
    if set(run_rows["position_index"].astype(int)) != expected_positions:
        raise ValueError(f"Run {run} does not contain positions 0 through 16␣
 ↪exactly once.")
    if run_rows["position_index"].duplicated().any():
        raise ValueError(f"Run {run} contains duplicate position indices.")

record_parts = []
for file_row in FILE_MANIFEST.itertuples(index=False):
    records = scan_vmd3_bin(file_row.path)
    records.insert(0, "run", file_row.run)
    records.insert(1, "acquisition_type", file_row.acquisition_type)
    records.insert(2, "position_index", file_row.position_index)
    records.insert(3, "encoder_count", file_row.encoder_count)
    records.insert(4, "filename", file_row.filename)
    records.insert(5, "path", file_row.path)
    records["done_value"] = pd.Series(pd.NA, index=records.index, dtype="Int64")

    for index in records.index[records["stream"] == "done"]:
        payload = read_record_payload(
            records.at[index, "path"],
            records.at[index, "payload_offset"],
```

```
            records.at[index, "payload_length"],
        )
        records.at[index, "done_value"] = decode_done(payload)
    record_parts.append(records)

RECORD_MANIFEST = pd.concat(record_parts, ignore_index=True)
for column in ("position_index", "encoder_count", "done_value"):
    RECORD_MANIFEST[column] = RECORD_MANIFEST[column].astype("Int64")

validate_record_manifest(RECORD_MANIFEST)
display(FILE_MANIFEST.groupby("run").agg(files=("filename", "size")))
display(RECORD_MANIFEST.groupby(["run", "stream"]).size().unstack(fill_value=0))
```

```
Record-manifest validation passed for all files.
Note: RADC, RFFT, and DONE arrival order was not required to form repeating
triplets.
```

| run | files |
|---|---|
| B | 17 |
| C | 17 |
| D | 17 |

| run | stream | done | radc | rfft |
|---|---|---|---|---|
| B | | 340 | 340 | 340 |
| C | | 340 | 340 | 340 |
| D | | 340 | 340 | 340 |

### 1.7 3. Rail geometry and run loading

Encoder counts are converted to physical rail coordinates by assigning the complete 0-to-800,000 count span to the measured 10 cm aperture. Each RX element's fixed offset inside the four-element array is added to the mechanical rail coordinate, producing a separate 17-position phase-center trajectory for every physical channel.

Raw runs are loaded, validated, processed, and released one at a time. This retains the exact processing from the exploratory notebook while reducing peak memory use compared with keeping Runs B, C, and D in memory simultaneously.

[4]:
```
def build_sar_position_table(
    file_manifest,
    runs=("B", "C"),
    aperture_length_m=0.10,
    center_position_index=8,
):
    """
    Build and validate the physical rail-position table.
```

```
    Encoder counts from every requested run must agree at
    every position index.
    """

    run_position_tables = {}

    for run in runs:
        run = run.upper()

        run_rows = (
            file_manifest.loc[
                file_manifest["run"] == run,
                [
                    "position_index",
                    "encoder_count",
                ],
            ]
            .drop_duplicates()
            .sort_values(
                "position_index"
            )
            .reset_index(
                drop=True
            )
        )

        if run_rows.empty:
            raise ValueError(
                f"No position records were found for Run {run}."
            )

        run_rows[
            "position_index"
        ] = run_rows[
            "position_index"
        ].astype(int)

        run_rows[
            "encoder_count"
        ] = run_rows[
            "encoder_count"
        ].astype(int)

        run_position_tables[
            run
        ] = run_rows
```

```
    # Use the first requested run as the reference position table.
    reference_run = runs[0].upper()

    reference_table = (
        run_position_tables[
            reference_run
        ].copy()
    )

    expected_position_indices = np.asarray(
        SAR_POSITION_INDICES,
        dtype=int,
    )

    if not np.array_equal(
        reference_table[
            "position_index"
        ].to_numpy(),
        expected_position_indices,
    ):
        raise ValueError(
            f"Run {reference_run} does not contain the "
            "expected ordered SAR position indices."
        )

    # Confirm that all requested runs use identical encoder counts.
    for run in runs[1:]:
        run = run.upper()

        comparison_table = (
            run_position_tables[
                run
            ]
        )

        if not np.array_equal(
            comparison_table[
                "position_index"
            ].to_numpy(),
            reference_table[
                "position_index"
            ].to_numpy(),
        ):
            raise ValueError(
                f"Run {run} position indices do not match "
                f"Run {reference_run}."
            )
```

```
        if not np.array_equal(
            comparison_table[
                "encoder_count"
            ].to_numpy(),
            reference_table[
                "encoder_count"
            ].to_numpy(),
        ):
            raise ValueError(
                f"Run {run} encoder counts do not match "
                f"Run {reference_run}."
            )

    center_rows = reference_table.loc[
        reference_table[
            "position_index"
        ]
        == int(
            center_position_index
        )
    ]

    if len(center_rows) != 1:
        raise ValueError(
            "The center position index does not identify "
            "exactly one encoder count."
        )

    center_encoder_count = int(
        center_rows.iloc[0][
            "encoder_count"
        ]
    )

    minimum_encoder_count = int(
        reference_table[
            "encoder_count"
        ].min()
    )

    maximum_encoder_count = int(
        reference_table[
            "encoder_count"
        ].max()
    )
```

```
    encoder_span = (
        maximum_encoder_count
        - minimum_encoder_count
    )

    if encoder_span <= 0:
        raise ValueError(
            "The encoder-count span must be positive."
        )

    meters_per_encoder_count = (
        aperture_length_m
        / encoder_span
    )

    reference_table[
        "rail_position_m"
    ] = (
        reference_table[
            "encoder_count"
        ]
        - center_encoder_count
    ) * meters_per_encoder_count

    reference_table[
        "rail_position_cm"
    ] = (
        100
        * reference_table[
            "rail_position_m"
        ]
    )

    rail_position_differences_m = np.diff(
        reference_table[
            "rail_position_m"
        ].to_numpy()
    )

    if not np.allclose(
        rail_position_differences_m,
        rail_position_differences_m[0],
    ):
        raise ValueError(
            "The calculated rail positions are not uniformly spaced."
        )
```

```
    reference_table.attrs[
        "aperture_length_m"
    ] = aperture_length_m

    reference_table.attrs[
        "center_position_index"
    ] = center_position_index

    reference_table.attrs[
        "center_encoder_count"
    ] = center_encoder_count

    reference_table.attrs[
        "meters_per_encoder_count"
    ] = meters_per_encoder_count

    return reference_table


def load_sar_radc_run(
    record_manifest,
    run,
    position_table,
):
    """
    Load all RADC and DONE measurements for one SAR run.

    Returns
    -------
    radc_stack
        Shape:
        (position, frame, sample, chirp, RX channel)

    done_stack
        Shape:
        (position, frame)
    """

    run = run.upper()

    radc_by_position = []
    done_by_position = []

    expected_frame_count = None

    for position_index in position_table[
        "position_index"
```

```
    ].astype(int):
        position_radc = load_radc_stack(
            record_manifest,
            run=run,
            position_index=position_index,
        )

        position_done = load_done_values(
            record_manifest,
            run=run,
            position_index=position_index,
        )

        if expected_frame_count is None:
            expected_frame_count = (
                position_radc.shape[0]
            )

        if (
            position_radc.shape[0]
            != expected_frame_count
        ):
            raise ValueError(
                f"Run {run}, position {position_index}: "
                "RADC frame count differs from the "
                "other positions."
            )

        if (
            position_done.shape[0]
            != expected_frame_count
        ):
            raise ValueError(
                f"Run {run}, position {position_index}: "
                "DONE count does not match the RADC count."
            )

        radc_by_position.append(
            position_radc
        )

        done_by_position.append(
            position_done
        )

    radc_stack = np.stack(
        radc_by_position,
```

```
        axis=0,
    )

    done_stack = np.stack(
        done_by_position,
        axis=0,
    )

    return (
        radc_stack,
        done_stack,
    )


def validate_sar_radc_run(
    radc_stack,
    done_stack,
    position_table,
    label,
):
    """
    Validate one complete multi-position RADC run and return
    a position-level summary table.
    """

    expected_number_of_positions = len(
        position_table
    )

    if radc_stack.ndim != 5:
        raise ValueError(
            f"{label} RADC must have shape "
            "(position, frame, sample, chirp, RX)."
        )

    if (
        radc_stack.shape[0]
        != expected_number_of_positions
    ):
        raise ValueError(
            f"{label} contains {radc_stack.shape[0]} "
            "positions; expected "
            f"{expected_number_of_positions}."
        )

    if (
        radc_stack.shape[2:]
```

```
        != RADC_FRAME_SHAPE
    ):
        raise ValueError(
            f"{label} has per-frame shape "
            f"{radc_stack.shape[2:]}; expected "
            f"{RADC_FRAME_SHAPE}."
        )

    expected_done_shape = (
        radc_stack.shape[0],
        radc_stack.shape[1],
    )

    if done_stack.shape != expected_done_shape:
        raise ValueError(
            f"{label} DONE shape is {done_stack.shape}; "
            f"expected {expected_done_shape}."
        )

    if not np.iscomplexobj(
        radc_stack
    ):
        raise ValueError(
            f"{label} RADC data are not complex."
        )

    if not np.all(
        np.isfinite(
            radc_stack
        )
    ):
        raise ValueError(
            f"{label} RADC data contain nonfinite values."
        )

    # DONE values must be consecutive within every file.
    done_differences = np.diff(
        done_stack.astype(
            np.int64
        ),
        axis=1,
    )

    if not np.all(
        done_differences == 1
    ):
        raise ValueError(
```

```
            f"{label} contains nonconsecutive DONE values."
        )

    validation_rows = []

    for position_array_index, position_row in enumerate(
        position_table.itertuples(
            index=False
        )
    ):
        position_radc = radc_stack[
            position_array_index
        ]

        position_rms = np.sqrt(
            np.mean(
                np.abs(
                    position_radc
                ) ** 2
            )
        )

        validation_rows.append(
            {
                "run": label,
                "position_index": int(
                    position_row.position_index
                ),
                "encoder_count": int(
                    position_row.encoder_count
                ),
                "rail_position_cm": float(
                    position_row.rail_position_cm
                ),
                "frame_count": int(
                    position_radc.shape[0]
                ),
                "done_first": int(
                    done_stack[
                        position_array_index,
                        0,
                    ]
                ),
                "done_last": int(
                    done_stack[
                        position_array_index,
                        -1,
```

```
                    ]
                ),
                "radc_rms": float(
                    position_rms
                ),
                "exact_zero_fraction": float(
                    np.mean(
                        position_radc == 0
                    )
                ),
            }
        )

    return pd.DataFrame(
        validation_rows
    )
```

[5]:
```
sar_position_table = build_sar_position_table(
    file_manifest=FILE_MANIFEST,
    runs=("B", "C", "D"),
    aperture_length_m=SAR_APERTURE_LENGTH_M,
    center_position_index=CENTER_POSITION_INDEX,
)

rail_positions_m = sar_position_table["rail_position_m"].to_numpy(dtype=float)
rail_step_m = float(np.diff(rail_positions_m)[0])
rx_positions_m = (
    np.arange(4) - (4 - 1) / 2
) * RX_ELEMENT_SPACING_M
rx_phase_center_positions_m = (
    rail_positions_m[:, np.newaxis]
    + rx_positions_m[np.newaxis, :]
)

assert rx_phase_center_positions_m.shape == (17, 4)

assert np.isclose(rail_positions_m[0], -0.05)
assert np.isclose(rail_positions_m[CENTER_POSITION_INDEX], 0.0)
assert np.isclose(rail_positions_m[-1], 0.05)
assert np.isclose(rail_step_m, 0.00625)

print(
    f"Rail geometry: {len(rail_positions_m)} positions, "
    f"{rail_step_m * 1000:.2f} mm step, "
    f"{(rail_positions_m[-1] - rail_positions_m[0]) * 100:.1f} cm aperture"
)
display(sar_position_table)
```

```
Rail geometry: 17 positions, 6.25 mm step, 10.0 cm aperture
```

| | position_index | encoder_count | rail_position_m | rail_position_cm |
|---|---|---|---|---|
| 0 | 0 | 0 | -0.05000 | -5.000 |
| 1 | 1 | 50000 | -0.04375 | -4.375 |
| 2 | 2 | 100000 | -0.03750 | -3.750 |
| 3 | 3 | 150000 | -0.03125 | -3.125 |
| 4 | 4 | 200000 | -0.02500 | -2.500 |
| 5 | 5 | 250000 | -0.01875 | -1.875 |
| 6 | 6 | 300000 | -0.01250 | -1.250 |
| 7 | 7 | 350000 | -0.00625 | -0.625 |
| 8 | 8 | 400000 | 0.00000 | 0.000 |
| 9 | 9 | 450000 | 0.00625 | 0.625 |
| 10 | 10 | 500000 | 0.01250 | 1.250 |
| 11 | 11 | 550000 | 0.01875 | 1.875 |
| 12 | 12 | 600000 | 0.02500 | 2.500 |
| 13 | 13 | 650000 | 0.03125 | 3.125 |
| 14 | 14 | 700000 | 0.03750 | 3.750 |
| 15 | 15 | 750000 | 0.04375 | 4.375 |
| 16 | 16 | 800000 | 0.05000 | 5.000 |

### 1.8 4. Range processing, zero Doppler, and coherent frame averaging

For each RADC frame, every RX channel is independently Hamming-weighted across the 128 measured fast-time samples and evaluated on a 1024-point range-FFT grid. A Hann window is applied across the 64 chirps, and the stationary zero-Doppler component is computed by complex summation. The 20 repeated zero-Doppler profiles at each rail position are then coherently averaged without combining receiver channels.

The repeated-frame coherence used in the code is

$$\eta_{\mathrm{frame}}(q) = \frac{\left|\sum_f X_f(q)\right|}{\sum_f |X_f(q)|},$$

where a value near one indicates that the repeated-frame phases are stable enough for coherent averaging.

[6]:
```
WINDOW_FUNCTIONS = {
    "rectangular": lambda length: np.ones(length),
    "hann": np.hanning,
    "hamming": np.hamming,
    "blackman": np.blackman,
}

def create_window(window_name, length):
    normalized_name = window_name.lower()
    if normalized_name not in WINDOW_FUNCTIONS:
        raise ValueError(
```

```
            f"Unsupported window '{window_name}'. "
            f"Choose from {list(WINDOW_FUNCTIONS)}."
        )
    return WINDOW_FUNCTIONS[normalized_name](length)


def process_all_rx_sar_with_oversampled_range_fft(
    sar_radc_stack,
    range_fft_size,
    range_window="hamming",
    doppler_window="hann",
):
    """Range-process all physical RX channels without combining them."""
    if sar_radc_stack.ndim != 5:
        raise ValueError(
            "sar_radc_stack must have shape "
            "(position, frame, sample, chirp, RX)."
        )

    (
        number_of_positions,
        number_of_frames,
        number_of_fast_time_samples,
        number_of_chirps,
        number_of_rx_channels,
    ) = sar_radc_stack.shape

    if range_fft_size < number_of_fast_time_samples:
        raise ValueError(
            "range_fft_size must be at least the number of measured samples."
        )

    range_coefficients = create_window(
        range_window, number_of_fast_time_samples
    )
    doppler_coefficients = create_window(
        doppler_window, number_of_chirps
    )

    # Output axes: position, repeated frame, range bin, RX channel.
    zero_doppler_stack = np.empty(
        (
            number_of_positions,
            number_of_frames,
            range_fft_size,
            number_of_rx_channels,
        ),
```

```
        dtype=np.complex128,
    )

    for position_index in range(number_of_positions):
        for frame_index in range(number_of_frames):
            # Input axes: measured sample, chirp, RX channel.
            radc_frame = np.asarray(
                sar_radc_stack[position_index, frame_index],
                dtype=np.complex128,
            )
            windowed_fast_time = (
                radc_frame
                * range_coefficients[:, np.newaxis, np.newaxis]
            )
            range_fft = np.fft.fft(
                windowed_fast_time,
                n=range_fft_size,
                axis=0,
            )
            windowed_chirps = (
                range_fft
                * doppler_coefficients[np.newaxis, :, np.newaxis]
            )
            # The complex chirp sum is the unshifted zero-Doppler bin.
            zero_doppler_stack[position_index, frame_index] = np.sum(
                windowed_chirps,
                axis=1,
            )

    return zero_doppler_stack, number_of_chirps // 2


def calculate_all_rx_frame_coherence(zero_doppler_stack):
    """Repeated-frame coherent-sum efficiency for every range bin and RX."""
    if zero_doppler_stack.ndim != 4:
        raise ValueError(
            "zero_doppler_stack must have shape "
            "(position, frame, range bin, RX)."
        )
    coherent_magnitude = np.abs(np.sum(zero_doppler_stack, axis=1))
    maximum_magnitude = np.sum(np.abs(zero_doppler_stack), axis=1)
    return coherent_magnitude / np.maximum(
        maximum_magnitude,
        np.finfo(float).tiny,
    )
```

[7]:
```
def load_validate_and_process_run(run):
    '''Load one rail run, validate it, and return all-RX range products.'''
    print(f"Loading and processing Run {run}...")
    radc_stack, done_stack = load_sar_radc_run(
        record_manifest=RECORD_MANIFEST,
        run=run,
        position_table=sar_position_table,
    )
    validation = validate_sar_radc_run(
        radc_stack=radc_stack,
        done_stack=done_stack,
        position_table=sar_position_table,
        label=run,
    )
    zero_doppler_stack, centered_zero_doppler_bin = (
        process_all_rx_sar_with_oversampled_range_fft(
            sar_radc_stack=radc_stack,
            range_fft_size=RANGE_FFT_SIZE,
            range_window=RANGE_WINDOW,
            doppler_window=DOPPLER_WINDOW,
        )
    )
    frame_coherence = calculate_all_rx_frame_coherence(zero_doppler_stack)
    coherent_mean = np.mean(zero_doppler_stack, axis=1)
    print(
        f"  raw shape = {radc_stack.shape}; "
        f"processed shape = {coherent_mean.shape}; "
        f"centered zero-Doppler bin = {centered_zero_doppler_bin}"
    )
    return {
        "coherent_mean": coherent_mean,
        "frame_coherence": frame_coherence,
        "validation": validation,
        "zero_doppler_bin": centered_zero_doppler_bin,
    }

RUN_PRODUCTS = {
    run: load_validate_and_process_run(run)
    for run in ("B", "C", "D")
}

validation_summary = pd.concat(
    [RUN_PRODUCTS[run]["validation"] for run in ("B", "C", "D")],
    ignore_index=True,
)
print("All three rail runs passed RADC and DONE validation.")
display(
```

```
    validation_summary.groupby("run").agg(
        positions=("position_index", "count"),
        frames_per_position=("frame_count", "median"),
        median_radc_rms=("radc_rms", "median"),
        maximum_zero_fraction=("exact_zero_fraction", "max"),
    )
)
```

```
Loading and processing Run B…
  raw shape = (17, 20, 128, 64, 4); processed shape = (17, 1024, 4); centered
zero-Doppler bin = 32
Loading and processing Run C…
  raw shape = (17, 20, 128, 64, 4); processed shape = (17, 1024, 4); centered
zero-Doppler bin = 32
Loading and processing Run D…
  raw shape = (17, 20, 128, 64, 4); processed shape = (17, 1024, 4); centered
zero-Doppler bin = 32
All three rail runs passed RADC and DONE validation.
```

| run | positions | frames_per_position | median_radc_rms | maximum_zero_fraction |
|---|---|---|---|---|
| B | 17 | 20.0 | 768.512521 | 0.000008 |
| C | 17 | 20.0 | 777.765076 | 0.000468 |
| D | 17 | 20.0 | 777.015560 | 0.000000 |

### 1.9 5. Matched complex background subtraction

Run B is aligned to each target run independently at every rail position and for every RX channel. Over background reference bins, the complex coefficient

$$\alpha_{m,\ell}(\delta) = \frac{\sum_{q\in\mathcal{Q}} T_{m,\ell}(q) B^{*}_{m,\ell}(q-\delta)}{\sum_{q\in\mathcal{Q}} |B_{m,\ell}(q-\delta)|^2}$$

minimizes the least-squares mismatch between target-run data $T_m$ and a fractionally shifted background $B_m$. The pipeline tests shifts $\delta$ from $-2$ to $+2$ oversampled bins and retains the shift with the highest complex correlation. Linear interpolation is applied separately to the real and imaginary components, so phase is preserved on the complex range grid. The residual is

$$G_{m,\ell}(q) = T_{m,\ell}(q) - \alpha_{m,\ell}(\hat{\delta}) B_{m,\ell}(q-\hat{\delta}).$$

The target-associated interval is excluded when estimating $\alpha$ and $\delta$, preventing the sphere response from being fitted into the background. The same masks, shift grid, and threshold are used for Runs C and D. The selected shifts are reported so solutions near the search boundary can be flagged for review.

[8]:
```
def shift_complex_range_profile(profile, shift_bins):
    """Translate a complex range profile by a fractional FFT-grid bin."""
    profile = np.asarray(profile, dtype=np.complex128)
```

```
    coordinate = np.arange(profile.size, dtype=float)
    source_coordinate = coordinate - float(shift_bins)
    shifted_real = np.interp(
        source_coordinate,
        coordinate,
        profile.real,
        left=0.0,
        right=0.0,
    )
    shifted_imag = np.interp(
        source_coordinate,
        coordinate,
        profile.imag,
        left=0.0,
        right=0.0,
    )
    return shifted_real + 1j * shifted_imag


def align_and_subtract_fractional_range_shift(
    target_mean,
    background_mean,
    target_coherence,
    background_coherence,
    reference_range_mask,
    maximum_shift_bins=2.0,
    number_of_shift_trials=81,
    minimum_reference_level_db=-35.0,
):
    """Align Run B by fractional range shift plus complex scale, then subtract.
    """
    if target_mean.shape != background_mean.shape or target_mean.ndim != 2:
        raise ValueError(
            "Target and background means must share shape (position, range bin).
    "
        )
    number_of_positions, number_of_range_bins = target_mean.shape
    if reference_range_mask.shape != (number_of_range_bins,):
        raise ValueError("reference_range_mask has the wrong shape.")

    shift_trials = np.linspace(
        -float(maximum_shift_bins),
        float(maximum_shift_bins),
        int(number_of_shift_trials),
    )
    minimum_level = 10.0 ** (minimum_reference_level_db / 20.0)
    aligned_background = np.empty_like(background_mean, dtype=np.complex128)
```

```
    residual = np.empty_like(target_mean, dtype=np.complex128)
    coefficients = np.empty(number_of_positions, dtype=np.complex128)
    selected_shifts = np.empty(number_of_positions, dtype=float)
    rows = []

    for position_index in range(number_of_positions):
        target_profile = target_mean[position_index]
        background_profile = background_mean[position_index]
        target_peak = np.max(np.abs(target_profile[reference_range_mask]))
        target_strong = np.abs(target_profile) >= minimum_level * target_peak

        best = None
        for shift_bins in shift_trials:
            shifted_background = shift_complex_range_profile(
                background_profile,
                shift_bins,
            )
            background_peak = np.max(
                np.abs(shifted_background[reference_range_mask])
            )
            background_strong = (
                np.abs(shifted_background) >= minimum_level * background_peak
            )
            valid = reference_range_mask & target_strong & background_strong
            if np.count_nonzero(valid) < 10:
                continue

            target_reference = target_profile[valid]
            background_reference = shifted_background[valid]
            numerator = np.sum(
                target_reference * np.conj(background_reference)
            )
            denominator = np.sum(np.abs(background_reference) ** 2)
            coefficient = numerator / np.maximum(
                denominator,
                np.finfo(float).tiny,
            )
            correlation = np.abs(numerator) / np.maximum(
                np.sqrt(
                    np.sum(np.abs(target_reference) ** 2) * denominator
                ),
                np.finfo(float).tiny,
            )
            if best is None or correlation > best["correlation"]:
                best = {
                    "shift_bins": float(shift_bins),
                    "shifted_background": shifted_background,
```

```
                    "valid": valid,
                    "coefficient": coefficient,
                    "correlation": float(correlation),
                }

        if best is None:
            raise ValueError(
                f"No valid background-alignment trial at position␣
↪{position_index}."
            )

        aligned_profile = best["coefficient"] * best["shifted_background"]
        residual_profile = target_profile - aligned_profile
        valid = best["valid"]
        reference_reduction_db = 10.0 * np.log10(
            np.maximum(
                np.sum(np.abs(target_profile[valid]) ** 2),
                np.finfo(float).tiny,
            )
            / np.maximum(
                np.sum(np.abs(residual_profile[valid]) ** 2),
                np.finfo(float).tiny,
            )
        )

        aligned_background[position_index] = aligned_profile
        residual[position_index] = residual_profile
        coefficients[position_index] = best["coefficient"]
        selected_shifts[position_index] = best["shift_bins"]
        rows.append(
            {
                "position_index": position_index,
                "selected_shift_oversampled_bins": best["shift_bins"],
                "alignment_magnitude_db": 20.0 * np.log10(
                    np.maximum(
                        np.abs(best["coefficient"]),
                        np.finfo(float).tiny,
                    )
                ),
                "alignment_phase_deg": np.rad2deg(
                    np.angle(best["coefficient"])
                ),
                "complex_correlation": best["correlation"],
                "median_target_frame_coherence": np.median(
                    target_coherence[position_index, valid]
                ),
                "median_background_frame_coherence": np.median(
```

```
                    background_coherence[position_index, valid]
                ),
                "reference_residual_reduction_db": reference_reduction_db,
            }
        )

    return (
        aligned_background,
        residual,
        coefficients,
        selected_shifts,
        pd.DataFrame(rows),
    )
```

[9]:
```
range_oversampling_factor = RANGE_FFT_SIZE / RADC_FRAME_SHAPE[0]
equivalent_original_bin = np.arange(RANGE_FFT_SIZE) / range_oversampling_factor

reference_min, reference_max = BACKGROUND_REFERENCE_ORIGINAL_BINS
exclusion_min, exclusion_max = BACKGROUND_EXCLUSION_ORIGINAL_BINS
background_reference_mask = (
    (equivalent_original_bin >= reference_min)
    & (equivalent_original_bin < reference_max)
    & ~(
        (equivalent_original_bin >= exclusion_min)
        & (equivalent_original_bin < exclusion_max)
    )
)

TARGET_PRODUCTS = {}
for run in ("C", "D"):
    profile_shape = RUN_PRODUCTS[run]["coherent_mean"].shape
    aligned_background = np.empty(profile_shape, dtype=np.complex128)
    residual = np.empty(profile_shape, dtype=np.complex128)
    alignment_coefficients = np.empty(
        (profile_shape[0], profile_shape[2]),
        dtype=np.complex128,
    )
    selected_background_shifts = np.empty(
        (profile_shape[0], profile_shape[2]),
        dtype=float,
    )
    channel_tables = []

    for rx_channel in RX_CHANNELS:
        (
            aligned_background[:, :, rx_channel],
            residual[:, :, rx_channel],
```

```
            alignment_coefficients[:, rx_channel],
            selected_background_shifts[:, rx_channel],
            channel_table,
        ) = align_and_subtract_fractional_range_shift(
            target_mean=RUN_PRODUCTS[run]["coherent_mean"][:, :, rx_channel],
            background_mean=RUN_PRODUCTS["B"]["coherent_mean"][:, :,␣
↪rx_channel],
            target_coherence=RUN_PRODUCTS[run]["frame_coherence"][:, :,␣
↪rx_channel],
            background_coherence=RUN_PRODUCTS["B"]["frame_coherence"][:, :,␣
↪rx_channel],
            reference_range_mask=background_reference_mask,
            maximum_shift_bins=BACKGROUND_SHIFT_SEARCH_BINS,
            number_of_shift_trials=BACKGROUND_SHIFT_TRIALS,
            minimum_reference_level_db=BACKGROUND_MINIMUM_REFERENCE_LEVEL_DB,
        )
        channel_table.insert(1, "rx_channel", int(rx_channel))
        channel_tables.append(channel_table)

    alignment_table = pd.concat(channel_tables, ignore_index=True)
    TARGET_PRODUCTS[run] = {
        "aligned_background": aligned_background,
        "residual": residual,
        "alignment_coefficients": alignment_coefficients,
        "selected_background_shifts": selected_background_shifts,
        "alignment_table": alignment_table,
    }

alignment_summary = pd.DataFrame(
    [
        {
            "run": run,
            "median_complex_correlation":␣
↪TARGET_PRODUCTS[run]["alignment_table"]["complex_correlation"].median(),
            "minimum_complex_correlation":␣
↪TARGET_PRODUCTS[run]["alignment_table"]["complex_correlation"].min(),
            "median_residual_reduction_db":␣
↪TARGET_PRODUCTS[run]["alignment_table"]["reference_residual_reduction_db"].
↪median(),
            "median_target_frame_coherence":␣
↪TARGET_PRODUCTS[run]["alignment_table"]["median_target_frame_coherence"].
↪median(),
            "median_abs_shift_oversampled_bins":␣
↪TARGET_PRODUCTS[run]["alignment_table"]["selected_shift_oversampled_bins"].
↪abs().median(),
        }
```

```
        for run in ("C", "D")
    ]
)
display(alignment_summary)
```

```
  run  median_complex_correlation  minimum_complex_correlation  \
0   C                    0.998575                     0.986972
1   D                    0.998844                     0.986207

   median_residual_reduction_db  median_target_frame_coherence  \
0                     25.458062                       0.999994
1                     26.364393                       0.999995

   median_abs_shift_oversampled_bins
0                                0.0
1                                0.0
```

### 1.10 6. V-MD3 absolute-range calibration

The source analysis found the laser-referenced sphere near original range bin $q \approx 22$. Under the nominal mapping $6/128 = 0.046875$ m per measured bin, that bin would be interpreted as approximately 1.031 m, while the laser-measured slant range was 0.84 m. The discrepancy is removed here with an explicit calibration model:

$$R_{\mathrm{cal},\ell}(q) = a_\ell q + b_\ell.$$

For the present one-point calibration, the nominal bin spacing fixes

$$a = \frac{6\ \mathrm{m}}{1024},$$

and the known laser slant range determines

$$b_\ell = R_{\mathrm{known}} - a_\ell q_{\mathrm{C},\ell}.$$

Backprojection uses the inverse mapping

$$q_\ell(R) = \frac{R - b_\ell}{a_\ell}$$

when interpolating each channel's complex range profile. Separate coefficients allow the current calibration to absorb small channel-dependent delay differences. This is mathematically equivalent to the provisional local registration in the source notebook, but it exposes the correction as a reusable calibration stage. The raw samples are not shifted or resampled in advance.

The current reference was acquired in Run C, but this is **not a Run C- or Run D-specific image adjustment**. The resulting V-MD3 calibration coefficients are applied to every dataset acquired with the same chirp and range settings. Run D's independent laser measurement is also 0.84 m

and is retained as a validation check rather than used to refit the correction. The calibration must be repeated if the waveform settings change.

**Important:** a single known range identifies only an additive bias while retaining the nominal scale. Two or more distinct calibration ranges are required to estimate both scale and intercept independently. A future dedicated calibration acquisition should collect known targets at multiple slant ranges. The function below supports that extension; with multiple reference measurements it fits both coefficients by least squares.

[10]:
```
def fit_range_calibration(measured_bins, known_ranges_m, nominal_spacing_m):
    '''Fit R_cal(q)=a*q+b; one point fits bias only, multiple points fit scale␣
 ↪and bias.'''
    measured_bins = np.atleast_1d(np.asarray(measured_bins, dtype=float))
    known_ranges_m = np.atleast_1d(np.asarray(known_ranges_m, dtype=float))
    if measured_bins.shape != known_ranges_m.shape or measured_bins.size == 0:
        raise ValueError("Calibration bins and ranges must be nonempty and␣
 ↪shape-matched.")

    if measured_bins.size == 1:
        slope_m_per_bin = float(nominal_spacing_m)
        intercept_m = float(known_ranges_m[0] - slope_m_per_bin *␣
 ↪measured_bins[0])
        method = "one-point additive-bias calibration"
    else:
        design = np.column_stack([measured_bins, np.ones(measured_bins.size)])
        slope_m_per_bin, intercept_m = np.linalg.lstsq(
            design, known_ranges_m, rcond=None
        )[0]
        method = "multi-point scale-and-bias calibration"

    fitted_ranges_m = slope_m_per_bin * measured_bins + intercept_m
    residuals_m = known_ranges_m - fitted_ranges_m
    return {
        "slope_m_per_bin": float(slope_m_per_bin),
        "intercept_m": float(intercept_m),
        "method": method,
        "residuals_m": residuals_m,
    }

def estimate_fractional_peak_bin(power_profile, search_indices):
    '''Estimate a spectral peak between grid samples using a log-power parabola.
 ↪'''
    power_profile = np.asarray(power_profile, dtype=float)
    search_indices = np.asarray(search_indices, dtype=int)
    integer_peak = int(
        search_indices[np.argmax(power_profile[search_indices])]
    )
    if integer_peak <= 0 or integer_peak >= power_profile.size - 1:
```

```
        return float(integer_peak)

    local_db = 10.0 * np.log10(
        np.maximum(
            power_profile[integer_peak - 1:integer_peak + 2],
            np.finfo(float).tiny,
        )
    )
    denominator = local_db[0] - 2.0 * local_db[1] + local_db[2]
    if np.isclose(denominator, 0.0):
        fractional_offset = 0.0
    else:
        fractional_offset = 0.5 * (local_db[0] - local_db[2]) / denominator
    fractional_offset = float(np.clip(fractional_offset, -0.5, 0.5))
    return float(integer_peak + fractional_offset)

target_min, target_max = TARGET_SEARCH_ORIGINAL_BINS
target_search_mask = (
    (equivalent_original_bin >= target_min)
    & (equivalent_original_bin <= target_max)
)
target_search_indices = np.flatnonzero(target_search_mask)

calibration_residual_power = np.sum(
    np.abs(TARGET_PRODUCTS[CALIBRATION_SOURCE_RUN]["residual"]) ** 2,
    axis=0,
)
laser_reference_slant_range_m = LASER_REFERENCE_SLANT_RANGE_M
nominal_oversampled_spacing_m = NOMINAL_MAXIMUM_RANGE_M / RANGE_FFT_SIZE

RANGE_CALIBRATION_BY_RX = {}
calibration_rows = []
for rx_channel in RX_CHANNELS:
    channel_reference_bin = estimate_fractional_peak_bin(
        calibration_residual_power[:, rx_channel],
        target_search_indices,
    )
    channel_calibration = fit_range_calibration(
        measured_bins=[channel_reference_bin],
        known_ranges_m=[laser_reference_slant_range_m],
        nominal_spacing_m=nominal_oversampled_spacing_m,
    )
    RANGE_CALIBRATION_BY_RX[int(rx_channel)] = channel_calibration

    nominal_range_m = channel_reference_bin * nominal_oversampled_spacing_m
    calibrated_range_m = (
        channel_calibration["slope_m_per_bin"] * channel_reference_bin
```

```
        + channel_calibration["intercept_m"]
    )
    calibration_rows.append(
        {
            "rx_channel": int(rx_channel),
            "method": channel_calibration["method"],
            "reference acquisition": f"Run {CALIBRATION_SOURCE_RUN}",
            "laser-reference oversampled bin": channel_reference_bin,
            "equivalent original bin": channel_reference_bin /␣
↪range_oversampling_factor,
            "nominal range at bin (m)": nominal_range_m,
            "laser slant range (m)": laser_reference_slant_range_m,
            "range intercept b (m)": channel_calibration["intercept_m"],
            "calibrated range at bin (m)": calibrated_range_m,
        }
    )

range_calibration_summary = pd.DataFrame(calibration_rows)
display(range_calibration_summary)
```

```
   rx_channel                                method reference acquisition  \
0           0  one-point additive-bias calibration                 Run C
1           1  one-point additive-bias calibration                 Run C
2           2  one-point additive-bias calibration                 Run C
3           3  one-point additive-bias calibration                 Run C

   laser-reference oversampled bin  equivalent original bin  \
0                       175.295680                21.911960
1                       175.213904                21.901738
2                       175.104467                21.888058
3                       175.545082                21.943135

   nominal range at bin (m)  laser slant range (m)  range intercept b (m)  \
0                  1.027123                   0.84              -0.187123
1                  1.026644                   0.84              -0.186644
2                  1.026003                   0.84              -0.186003
3                  1.028584                   0.84              -0.188584

   calibrated range at bin (m)
0                         0.84
1                         0.84
2                         0.84
3                         0.84
```

## 1.11 7. Calibrated near-field backprojection

For each candidate pixel $(x, z)$ and aperture phase center $x_m$, the monostatic distance is

$$R_m(x,z) = \sqrt{(x - x_m)^2 + z^2}.$$

The calibrated inverse range mapping identifies the fractional FFT coordinate to interpolate. The measured rail-dependent phase is then removed with

$$\exp\left[-j\, s_\phi \frac{4\pi}{\lambda}\left(R_m - R_{\text{center}}\right)\right],$$

where the validated V-MD3 phase sign is $s_\phi = +1$. Hamming weights are applied across the 17 mechanical positions before coherent summation.

All four RX channels are backprojected independently. For channel $\ell$, the phase-center trajectory is the 17-position rail coordinate plus that RX element's fixed array offset. The propagation model remains a monostatic approximation because the exact V-MD3 TX phase center has not yet been independently measured.

### 1.11.1 Four-RX fusion modes

Let $I_\ell(x,z)$ be the complex SAR image formed from RX channel $\ell$. The baseline equal-power image is

$$P_{\text{equal}}(x,z) = \frac{1}{4}\sum_{\ell=0}^{3} \left|I_\ell(x,z)\right|^2.$$

This is called **power fusion** because magnitude-squared values are added after each RX has been coherently focused over the 17 rail positions. A real target that appears consistently in several channels contributes power repeatedly at the same image location, while uncorrelated noise and some channel-specific artifacts are less likely to reinforce in the same way. Power fusion therefore improves robustness and can improve effective signal-to-noise ratio.

The enhanced pipeline also reports a quality-weighted power image,

$$P_{\text{weighted}}(x,z) = \sum_{\ell=0}^{3} w_\ell \left|I_\ell(x,z)\right|^2, \qquad \sum_\ell w_\ell = 1,$$

where the fixed $w_\ell$ values are derived only from Run C and capped at a 4:1 spread. Both power-fusion paths intentionally discard the phase relationship between $I_0$, $I_1$, $I_2$, and $I_3$, so unknown fixed channel phases cannot defocus the combined result.

The notebook separately tests the coherent field sum $\left|\sum_\ell a_\ell c_\ell I_\ell\right|^2$, using Run C-derived phase factors $c_\ell$ and amplitude weights $a_\ell = \sqrt{w_\ell}$. This may sharpen a consistent target response, but an incorrect or non-transferable inter-channel phase estimate can also create cancellation or false structure. It therefore remains an experimental comparison.

[11]:
```
def backproject_calibrated_rx_channel(
    aperture_range_profiles,
    aperture_positions_m,
    image_cross_range_m,
```

```
    image_downrange_m,
    carrier_frequency_hz,
    measured_phase_sign,
    range_slope_m_per_bin,
    range_intercept_m,
    reference_aperture_index,
    aperture_weights,
):
    '''Form a calibrated single-RX near-field SAR image by complex␣
 ↪backprojection.'''
    profiles = np.asarray(aperture_range_profiles, dtype=np.complex128)
    aperture_positions_m = np.asarray(aperture_positions_m, dtype=float)
    weights = np.asarray(aperture_weights, dtype=float)
    if profiles.ndim != 2:
        raise ValueError("Profiles must have shape (aperture position, range␣
 ↪bin).")
    if aperture_positions_m.shape != (profiles.shape[0],):
        raise ValueError("Aperture positions do not match the profile stack.")
    if weights.shape != (profiles.shape[0],) or np.any(weights < 0):
        raise ValueError("Aperture weights are invalid.")
    weights = weights / np.sum(weights)

    wavelength_m = 299_792_458.0 / carrier_frequency_hz
    cross_grid_m, down_grid_m = np.meshgrid(
        image_cross_range_m, image_downrange_m
    )
    coherent_image = np.zeros(cross_grid_m.shape, dtype=np.complex128)
    noncoherent_sum = np.zeros(cross_grid_m.shape, dtype=float)
    valid_support_count = np.zeros(cross_grid_m.shape, dtype=np.int32)

    center_position_m = aperture_positions_m[reference_aperture_index]
    center_pixel_range_m = np.sqrt(
        (cross_grid_m - center_position_m) ** 2 + down_grid_m ** 2
    )

    for aperture_index, aperture_position_m in enumerate(aperture_positions_m):
        pixel_range_m = np.sqrt(
            (cross_grid_m - aperture_position_m) ** 2 + down_grid_m ** 2
        )
        fractional_bin = (pixel_range_m - range_intercept_m) /␣
 ↪range_slope_m_per_bin
        lower_bin = np.floor(fractional_bin).astype(np.int64)
        fraction = fractional_bin - lower_bin
        valid = (lower_bin >= 0) & (lower_bin < profiles.shape[1] - 1)
        safe_lower = np.clip(lower_bin, 0, profiles.shape[1] - 2)
        lower_sample = profiles[aperture_index, safe_lower]
        upper_sample = profiles[aperture_index, safe_lower + 1]
```

```
        interpolated = (1.0 - fraction) * lower_sample + fraction * upper_sample
        interpolated = np.where(valid, interpolated, 0.0 + 0.0j)

        relative_range_m = pixel_range_m - center_pixel_range_m
        phase_correction = np.exp(
            -1j
            * measured_phase_sign
            * 4.0
            * np.pi
            / wavelength_m
            * relative_range_m
        )
        coherent_image += weights[aperture_index] * interpolated *␣
 ↪phase_correction
        noncoherent_sum += weights[aperture_index] * np.abs(interpolated)
        valid_support_count += valid.astype(np.int32)

    focusing_coherence = np.abs(coherent_image) / np.maximum(
        noncoherent_sum, np.finfo(float).tiny
    )
    return coherent_image, focusing_coherence, valid_support_count

def normalized_power_db(
    complex_or_magnitude_data,
    minimum_db=-80.0,
):
    """
    Convert a complex image or a nonnegative magnitude image
    into normalized power in decibels.

    The result is normalized to its own maximum, so the
    strongest displayed pixel is 0 dB.
    """

    # abs() returns magnitude for complex data and leaves a
    # nonnegative magnitude image unchanged.
    magnitude = np.abs(
        complex_or_magnitude_data
    )

    # Radar images are displayed here as normalized power.
    power = magnitude ** 2

    normalized_power = (
        power
        / np.maximum(
            np.max(
```

```
                power
            ),
            np.finfo(float).tiny,
        )
    )

    power_db = (
        10
        * np.log10(
            np.maximum(
                normalized_power,
                np.finfo(float).tiny,
            )
        )
    )

    # Apply a lower display floor so extremely small numerical
    # values do not dominate the plot's color scale.
    return np.maximum(
        power_db,
        minimum_db,
    )


def measure_contiguous_minus_3db_width(
    coordinate,
    power_profile,
):
    """
    Measure the contiguous -3 dB width surrounding the
    strongest peak in a one-dimensional power profile.

    This measures the width of the observed image response.
    For an extended sphere, it should not automatically be
    interpreted as the point-target resolution.
    """

    coordinate = np.asarray(
        coordinate,
        dtype=float,
    )

    power_profile = np.asarray(
        power_profile,
        dtype=float,
    )
```

```
    if coordinate.ndim != 1:
        raise ValueError(
            "coordinate must be one-dimensional."
        )

    if power_profile.shape != coordinate.shape:
        raise ValueError(
            "power_profile must match coordinate."
        )

    # Normalize the profile so that its strongest point is
    # exactly 0 dB.
    normalized_power = (
        power_profile
        / np.maximum(
            np.max(
                power_profile
            ),
            np.finfo(float).tiny,
        )
    )

    power_profile_db = (
        10
        * np.log10(
            np.maximum(
                normalized_power,
                np.finfo(float).tiny,
            )
        )
    )

    # Identify the strongest point in the profile.
    peak_index = int(
        np.argmax(
            power_profile
        )
    )

    threshold_db = -3.0

    # Starting at the peak, walk left until the profile falls
    # below -3 dB. This keeps the measurement attached to the
    # main peak instead of accidentally including a separated
    # sidelobe above -3 dB.
    left_inside_index = peak_index
```

```
    while (
        left_inside_index > 0
        and power_profile_db[
            left_inside_index - 1
        ]
        >= threshold_db
    ):
        left_inside_index -= 1

    # Repeat the same search toward increasing coordinate.
    right_inside_index = peak_index

    while (
        right_inside_index
        < coordinate.size - 1
        and power_profile_db[
            right_inside_index + 1
        ]
        >= threshold_db
    ):
        right_inside_index += 1

    def interpolate_threshold_crossing(
        coordinate_1,
        level_1,
        coordinate_2,
        level_2,
        threshold,
    ):
        """
        Estimate the coordinate where two adjacent samples
        cross the selected dB threshold.
        """

        if np.isclose(
            level_1,
            level_2,
        ):
            return (
                coordinate_1
                + coordinate_2
            ) / 2

        return (
            coordinate_1
            + (
                threshold
```

```
            - level_1
        )
        * (
            coordinate_2
            - coordinate_1
        )
        / (
            level_2
            - level_1
        )
    )

# Interpolate the left -3 dB crossing. If the profile
# remains above -3 dB at the grid boundary, the width
# cannot be measured completely.
if left_inside_index == 0:
    left_crossing = np.nan
else:
    left_crossing = (
        interpolate_threshold_crossing(
            coordinate[
                left_inside_index - 1
            ],
            power_profile_db[
                left_inside_index - 1
            ],
            coordinate[
                left_inside_index
            ],
            power_profile_db[
                left_inside_index
            ],
            threshold_db,
        )
    )

# Interpolate the right -3 dB crossing.
if (
    right_inside_index
    == coordinate.size - 1
):
    right_crossing = np.nan
else:
    right_crossing = (
        interpolate_threshold_crossing(
            coordinate[
                right_inside_index
```

```
                ],
                power_profile_db[
                    right_inside_index
                ],
                coordinate[
                    right_inside_index + 1
                ],
                power_profile_db[
                    right_inside_index + 1
                ],
                threshold_db,
            )
        )

    # A valid width requires both threshold crossings.
    if (
        np.isfinite(
            left_crossing
        )
        and np.isfinite(
            right_crossing
        )
    ):
        width = (
            right_crossing
            - left_crossing
        )
    else:
        width = np.nan

    return {
        "peak_index": peak_index,
        "peak_coordinate": coordinate[
            peak_index
        ],
        "left_crossing": left_crossing,
        "right_crossing": right_crossing,
        "width": width,
        "power_profile_db": power_profile_db,
    }


def create_synthetic_aperture_weights(
    window_name,
    number_of_positions,
):
    """
```

```
    Create normalized nonnegative amplitude weights for the
    synthetic-aperture positions.

    The returned weights sum to one so that changing the window
    does not introduce an arbitrary coherent-image scale.
    """

    normalized_name = window_name.lower()

    if normalized_name == "uniform":
        raw_weights = np.ones(
            number_of_positions,
            dtype=float,
        )

    elif normalized_name == "hamming":
        raw_weights = np.hamming(
            number_of_positions
        )

    elif normalized_name == "hann":
        raw_weights = np.hanning(
            number_of_positions
        )

    else:
        raise ValueError(
            "Unsupported aperture window. Choose from "
            "'uniform', 'hamming', or 'hann'."
        )

    if not np.any(
        raw_weights > 0
    ):
        raise ValueError(
            "The aperture window contains no positive weights."
        )

    # Normalize the coefficients to unit sum.
    #
    # This preserves image shape and relative coherent gain
    # comparisons without allowing a window with a larger
    # coefficient sum to appear artificially brighter.
    normalized_weights = (
        raw_weights
        / np.sum(
            raw_weights
```

```
        )
    )

    return normalized_weights


def characterize_mainlobe_and_outside_response(
    coordinate,
    power_profile,
):
    """
    Characterize one normalized power profile.

    The nearest local minima around the strongest peak define
    the central main-lobe boundaries. The largest value outside
    those boundaries is reported as an out-of-main-lobe level.

    For ideal point-target simulations, this is the ideal peak
    sidelobe level.

    For measured data, it may also contain multipath, clutter,
    or additional scattering centers.
    """

    coordinate = np.asarray(
        coordinate,
        dtype=float,
    )

    power_profile = np.asarray(
        power_profile,
        dtype=float,
    )

    if coordinate.shape != power_profile.shape:
        raise ValueError(
            "coordinate and power_profile must match."
        )

    normalized_power = (
        power_profile
        / np.maximum(
            np.max(
                power_profile
            ),
            np.finfo(float).tiny,
        )
```

```
    )

    normalized_power_db = (
        10
        * np.log10(
            np.maximum(
                normalized_power,
                np.finfo(float).tiny,
            )
        )
    )

    peak_index = int(
        np.argmax(
            normalized_power
        )
    )

    # Measure the contiguous -3 dB width surrounding the
    # strongest peak.
    minus3db_result = (
        measure_contiguous_minus_3db_width(
            coordinate=coordinate,
            power_profile=normalized_power,
        )
    )

    # Find local minima that can serve as the boundaries of
    # the central main lobe.
    local_minimum_indices = (
        np.flatnonzero(
            (
                normalized_power[1:-1]
                <= normalized_power[:-2]
            )
            & (
                normalized_power[1:-1]
                < normalized_power[2:]
            )
        )
        + 1
    )

    left_minimum_candidates = (
        local_minimum_indices[
            local_minimum_indices
            < peak_index
```

```
        ]
    )

    right_minimum_candidates = (
        local_minimum_indices[
            local_minimum_indices
            > peak_index
        ]
    )

    if (
        left_minimum_candidates.size == 0
        or right_minimum_candidates.size == 0
    ):
        left_mainlobe_minimum_index = None
        right_mainlobe_minimum_index = None
        null_to_null_width = np.nan
        maximum_outside_mainlobe_db = np.nan

    else:
        left_mainlobe_minimum_index = (
            left_minimum_candidates[-1]
        )

        right_mainlobe_minimum_index = (
            right_minimum_candidates[0]
        )

        null_to_null_width = (
            coordinate[
                right_mainlobe_minimum_index
            ]
            - coordinate[
                left_mainlobe_minimum_index
            ]
        )

        # Exclude the region between the first minima and find
        # the largest remaining response.
        outside_mainlobe_mask = np.ones(
            normalized_power.shape,
            dtype=bool,
        )

        outside_mainlobe_mask[
            left_mainlobe_minimum_index:
            right_mainlobe_minimum_index + 1
```

```
        ] = False

        maximum_outside_mainlobe_db = np.max(
            normalized_power_db[
                outside_mainlobe_mask
            ]
        )

    return {
        "peak_index": peak_index,
        "peak_coordinate": coordinate[
            peak_index
        ],
        "minus3db_width": (
            minus3db_result[
                "width"
            ]
        ),
        "null_to_null_width": (
            null_to_null_width
        ),
        "maximum_outside_mainlobe_db": (
            maximum_outside_mainlobe_db
        ),
        "power_profile_db": (
            normalized_power_db
        ),
        "left_mainlobe_minimum_index": (
            left_mainlobe_minimum_index
        ),
        "right_mainlobe_minimum_index": (
            right_mainlobe_minimum_index
        ),
    }
```

[12]:
```
aperture_weights = create_synthetic_aperture_weights(
    window_name=APERTURE_WINDOW,
    number_of_positions=rx_phase_center_positions_m.shape[0],
)

def normalized_nonnegative_power_db(power, minimum_db=-60.0):
    '''Normalize an already-formed nonnegative power image to 0 dB.'''
    power = np.asarray(power, dtype=float)
    normalized = power / np.maximum(np.max(power), np.finfo(float).tiny)
    power_db = 10.0 * np.log10(
        np.maximum(normalized, np.finfo(float).tiny)
    )
```

```
    return np.maximum(power_db, minimum_db)

def find_peak_in_metric_region(power):
    '''Locate the strongest pixel only inside the interpretable target region.
 ↪'''
    cross_mask = np.abs(IMAGE_CROSS_RANGE_M) <= METRIC_SEARCH_CROSS_RANGE_M
    down_mask = (
        (IMAGE_DOWNRANGE_M >= METRIC_SEARCH_DOWNRANGE_M[0])
        & (IMAGE_DOWNRANGE_M <= METRIC_SEARCH_DOWNRANGE_M[1])
    )
    region_mask = down_mask[:, np.newaxis] & cross_mask[np.newaxis, :]
    restricted = np.where(region_mask, power, -np.inf)
    return np.unravel_index(np.argmax(restricted), power.shape)

def interpolate_complex_rows(profiles, fractional_bins):
    '''Interpolate one complex range sample from every aperture position.'''
    samples = np.empty(profiles.shape[0], dtype=np.complex128)
    coordinate = np.arange(profiles.shape[1], dtype=float)
    for position_index, fractional_bin in enumerate(fractional_bins):
        samples[position_index] = (
            np.interp(fractional_bin, coordinate, profiles[position_index].real)
            + 1j * np.interp(
                fractional_bin, coordinate, profiles[position_index].imag
            )
        )
    return samples

def form_per_rx_images(run, aperture_phase_corrections=None):
    '''Backproject all four RX channels, optionally after phase-screen␣
 ↪correction.'''
    channel_images = []
    channel_coherence_maps = []
    for rx_channel in RX_CHANNELS:
        channel_calibration = RANGE_CALIBRATION_BY_RX[int(rx_channel)]
        profiles = TARGET_PRODUCTS[run]["residual"][:, :, rx_channel]
        if aperture_phase_corrections is not None:
            profiles = (
                profiles
                * aperture_phase_corrections[:, rx_channel, np.newaxis]
            )
        image, focusing_coherence, support_count = (
            backproject_calibrated_rx_channel(
                aperture_range_profiles=profiles,
                aperture_positions_m=rx_phase_center_positions_m[:, rx_channel],
                image_cross_range_m=IMAGE_CROSS_RANGE_M,
                image_downrange_m=IMAGE_DOWNRANGE_M,
                carrier_frequency_hz=CARRIER_FREQUENCY_HZ,
```

```
                measured_phase_sign=MEASURED_PHASE_SIGN,
                range_slope_m_per_bin=channel_calibration["slope_m_per_bin"],
                range_intercept_m=channel_calibration["intercept_m"],
                reference_aperture_index=CENTER_POSITION_INDEX,
                aperture_weights=aperture_weights,
            )
        )
        if not np.all(
            support_count == rx_phase_center_positions_m.shape[0]
        ):
            raise ValueError(
                f"Run {run}, RX {rx_channel} has incomplete aperture support."
            )
        channel_images.append(image)
        channel_coherence_maps.append(focusing_coherence)
    return np.stack(channel_images), np.stack(channel_coherence_maps)

def estimate_smooth_aperture_phase_correction(
    profiles,
    aperture_positions_m,
    calibration_cross_range_m,
    calibration_downrange_m,
    range_calibration,
    polynomial_degree,
):
    '''Fit a smooth position-dependent phase error from the Run C target.'''
    wavelength_m = 299_792_458.0 / CARRIER_FREQUENCY_HZ
    pixel_ranges_m = np.sqrt(
        (calibration_cross_range_m - aperture_positions_m) ** 2
        + calibration_downrange_m ** 2
    )
    fractional_bins = (
        pixel_ranges_m - range_calibration["intercept_m"]
    ) / range_calibration["slope_m_per_bin"]
    measured_samples = interpolate_complex_rows(profiles, fractional_bins)
    relative_ranges_m = (
        pixel_ranges_m - pixel_ranges_m[CENTER_POSITION_INDEX]
    )
    geometric_correction = np.exp(
        -1j
        * MEASURED_PHASE_SIGN
        * 4.0
        * np.pi
        / wavelength_m
        * relative_ranges_m
    )
    compensated_samples = measured_samples * geometric_correction
```

```
    residual_phase_rad = np.unwrap(
        np.angle(
            compensated_samples
            * np.conj(compensated_samples[CENTER_POSITION_INDEX])
        )
    )

    normalized_position = aperture_positions_m / np.max(
        np.abs(aperture_positions_m)
    )
    design = np.vander(
        normalized_position,
        N=polynomial_degree + 1,
        increasing=True,
    )
    fit_weights = np.sqrt(
        np.abs(measured_samples)
        / np.maximum(np.max(np.abs(measured_samples)), np.finfo(float).tiny)
    )
    weighted_design = design * fit_weights[:, np.newaxis]
    weighted_phase = residual_phase_rad * fit_weights
    coefficients = np.linalg.lstsq(
        weighted_design,
        weighted_phase,
        rcond=None,
    )[0]
    fitted_phase_rad = design @ coefficients
    fitted_phase_rad -= fitted_phase_rad[CENTER_POSITION_INDEX]
    correction = np.exp(-1j * fitted_phase_rad)

    before_error = residual_phase_rad
    after_error = residual_phase_rad - fitted_phase_rad
    before_rms_deg = np.rad2deg(
        np.sqrt(np.average(before_error ** 2, weights=fit_weights ** 2))
    )
    after_rms_deg = np.rad2deg(
        np.sqrt(np.average(after_error ** 2, weights=fit_weights ** 2))
    )
    return correction, before_rms_deg, after_rms_deg, fitted_phase_rad

def calculate_rx_quality_weights():
    '''Derive bounded RX weights from Run C only, then freeze them.'''
    rows = []
    raw_scores = []
    for rx_channel in RX_CHANNELS:
        residual_power = np.sum(
            np.abs(
```

```
                TARGET_PRODUCTS[CALIBRATION_SOURCE_RUN]["residual"][
                    :, :, rx_channel
                ]
            ) ** 2,
            axis=0,
        )
        calibration_bin = range_calibration_summary.loc[
            range_calibration_summary["rx_channel"] == rx_channel,
            "laser-reference oversampled bin",
        ].iloc[0]
        target_power = np.interp(
            calibration_bin,
            np.arange(residual_power.size),
            residual_power,
        )
        reference_floor = np.median(
            residual_power[background_reference_mask]
        )
        target_to_reference_db = 10.0 * np.log10(
            np.maximum(target_power, np.finfo(float).tiny)
            / np.maximum(reference_floor, np.finfo(float).tiny)
        )
        nearest_bin = int(np.clip(np.round(calibration_bin), 0, RANGE_FFT_SIZE␣
↪- 1))
        frame_coherence = np.median(
            RUN_PRODUCTS[CALIBRATION_SOURCE_RUN]["frame_coherence"][
                :, nearest_bin, rx_channel
            ]
        )
        alignment_correlation = TARGET_PRODUCTS[
            CALIBRATION_SOURCE_RUN
        ]["alignment_table"].loc[
            lambda table: table["rx_channel"] == rx_channel,
            "complex_correlation",
        ].median()
        raw_score = (
            np.sqrt(
                np.maximum(target_power, np.finfo(float).tiny)
                / np.maximum(reference_floor, np.finfo(float).tiny)
            )
            * frame_coherence
            * alignment_correlation
        )
        raw_scores.append(raw_score)
        rows.append(
            {
                "rx_channel": int(rx_channel),
```

```
                "target_to_reference_db": target_to_reference_db,
                "frame_coherence": frame_coherence,
                "background_correlation": alignment_correlation,
                "raw_quality_score": raw_score,
            }
        )

    raw_scores = np.asarray(raw_scores, dtype=float)
    median_score = np.median(raw_scores)
    ratio_root = np.sqrt(RX_WEIGHT_MAX_TO_MIN_RATIO)
    bounded_scores = np.clip(
        raw_scores,
        median_score / ratio_root,
        median_score * ratio_root,
    )
    weights = bounded_scores / np.sum(bounded_scores)
    for row, weight in zip(rows, weights):
        row["fusion_weight"] = weight
    return weights, pd.DataFrame(rows)

def summarize_image_power(run, processing_path, power, channel_images,␣
 ↪coherence_maps):
    '''Measure one fused result inside the defined target-search region.'''
    peak_down, peak_cross = find_peak_in_metric_region(power)
    peak_x = IMAGE_CROSS_RANGE_M[peak_cross]
    peak_z = IMAGE_DOWNRANGE_M[peak_down]
    slant_range_m = np.sqrt(peak_x ** 2 + peak_z ** 2)
    cross_mask = np.abs(IMAGE_CROSS_RANGE_M) <= METRIC_SEARCH_CROSS_RANGE_M
    cross_characteristics = characterize_mainlobe_and_outside_response(
        IMAGE_CROSS_RANGE_M[cross_mask],
        power[peak_down, cross_mask],
    )
    down_width = measure_contiguous_minus_3db_width(
        IMAGE_DOWNRANGE_M,
        power[:, peak_cross],
    )
    left = cross_characteristics["left_mainlobe_minimum_index"]
    right = cross_characteristics["right_mainlobe_minimum_index"]
    cross_power = power[peak_down, cross_mask]
    if left is None or right is None:
        integrated_sidelobe_ratio_db = np.nan
    else:
        main_power = np.sum(cross_power[left:right + 1])
        outside_power = np.sum(cross_power) - main_power
        integrated_sidelobe_ratio_db = 10.0 * np.log10(
            np.maximum(outside_power, np.finfo(float).tiny)
            / np.maximum(main_power, np.finfo(float).tiny)
```

```
        )

    channel_peak_power = np.abs(
        channel_images[:, peak_down, peak_cross]
    ) ** 2
    supporting_coherence = np.average(
        coherence_maps[:, peak_down, peak_cross],
        weights=np.maximum(channel_peak_power, np.finfo(float).tiny),
    )
    result = {
        "power": power,
        "power_db": normalized_nonnegative_power_db(power),
        "peak_cross_range_m": peak_x,
        "peak_downrange_m": peak_z,
        "cross_range_profile_db": cross_characteristics["power_profile_db"],
        "cross_range_profile_coordinate_m": IMAGE_CROSS_RANGE_M[cross_mask],
    }
    row = {
        "run": run,
        "processing_path": processing_path,
        "peak_cross_range_m": peak_x,
        "peak_downrange_m": peak_z,
        "peak_center_slant_range_m": slant_range_m,
        "laser_slant_range_error_m": (
            slant_range_m - EXPECTED_TARGET_SLANT_RANGE_M[run]
        ),
        "supporting_rx_coherence": supporting_coherence,
        "minus3db_cross_range_width_m": cross_characteristics["minus3db_width"],
        "minus3db_downrange_width_m": down_width["width"],
        "peak_sidelobe_level_db": cross_characteristics[
            "maximum_outside_mainlobe_db"
        ],
        "integrated_sidelobe_ratio_db": integrated_sidelobe_ratio_db,
    }
    return result, row

# Stage 1: defined reference baseline per-RX images.
BASELINE_PER_RX = {}
for run in ("C", "D"):
    BASELINE_PER_RX[run] = form_per_rx_images(run)

baseline_c_power = np.mean(np.abs(BASELINE_PER_RX["C"][0]) ** 2, axis=0)
calibration_peak_down, calibration_peak_cross = find_peak_in_metric_region(
    baseline_c_power
)
calibration_peak_x_m = IMAGE_CROSS_RANGE_M[calibration_peak_cross]
calibration_peak_z_m = IMAGE_DOWNRANGE_M[calibration_peak_down]
```

```
# Stage 2: smooth Run C-derived aperture phase correction for every RX.
APERTURE_PHASE_CORRECTIONS = np.empty(
    (rx_phase_center_positions_m.shape[0], len(RX_CHANNELS)),
    dtype=np.complex128,
)
phase_calibration_rows = []
for rx_channel in RX_CHANNELS:
    correction, before_rms, after_rms, fitted_phase = (
        estimate_smooth_aperture_phase_correction(
            profiles=TARGET_PRODUCTS[CALIBRATION_SOURCE_RUN]["residual"][
                :, :, rx_channel
            ],
            aperture_positions_m=rx_phase_center_positions_m[:, rx_channel],
            calibration_cross_range_m=calibration_peak_x_m,
            calibration_downrange_m=calibration_peak_z_m,
            range_calibration=RANGE_CALIBRATION_BY_RX[int(rx_channel)],
            polynomial_degree=APERTURE_PHASE_POLYNOMIAL_DEGREE,
        )
    )
    APERTURE_PHASE_CORRECTIONS[:, rx_channel] = correction
    phase_calibration_rows.append(
        {
            "rx_channel": int(rx_channel),
            "phase_rms_before_deg": before_rms,
            "phase_rms_after_deg": after_rms,
            "maximum_applied_correction_deg": np.max(
                np.abs(np.rad2deg(fitted_phase))
            ),
        }
    )
PHASE_CALIBRATION_SUMMARY = pd.DataFrame(phase_calibration_rows)

# Stage 3: apply the frozen phase screen to both Run C and Run D.
CALIBRATED_PER_RX = {}
for run in ("C", "D"):
    CALIBRATED_PER_RX[run] = form_per_rx_images(
        run,
        aperture_phase_corrections=APERTURE_PHASE_CORRECTIONS,
    )

# Stage 4: derive bounded channel weights using Run C only.
RX_QUALITY_WEIGHTS, RX_QUALITY_TABLE = calculate_rx_quality_weights()

# Stage 5: estimate constant inter-RX phase corrections at Run C's
# calibrated weighted-power peak, then freeze them for Run D.
calibrated_c_images = CALIBRATED_PER_RX["C"][0]
```

```
preliminary_weighted_c_power = np.sum(
    RX_QUALITY_WEIGHTS[:, np.newaxis, np.newaxis]
    * np.abs(calibrated_c_images) ** 2,
    axis=0,
)
coherent_peak_down, coherent_peak_cross = find_peak_in_metric_region(
    preliminary_weighted_c_power
)
run_c_channel_values = calibrated_c_images[
    :, coherent_peak_down, coherent_peak_cross
]
INTER_RX_PHASE_CORRECTIONS = np.exp(
    -1j
    * (
        np.angle(run_c_channel_values)
        - np.angle(run_c_channel_values[0])
    )
)
coherent_amplitude_weights = np.sqrt(RX_QUALITY_WEIGHTS)
coherent_amplitude_weights /= np.sum(coherent_amplitude_weights)

# Stage 6: create baseline, enhanced power-fused, and experimental
# coherent products for both runs, then calculate common metrics.
PIPELINE_RESULTS = {"C": {}, "D": {}}
metric_rows = []
for run in ("C", "D"):
    baseline_images, baseline_coherence = BASELINE_PER_RX[run]
    calibrated_images, calibrated_coherence = CALIBRATED_PER_RX[run]

    baseline_equal_power = np.mean(np.abs(baseline_images) ** 2, axis=0)
    weighted_power = np.sum(
        RX_QUALITY_WEIGHTS[:, np.newaxis, np.newaxis]
        * np.abs(calibrated_images) ** 2,
        axis=0,
    )
    coherent_image = np.sum(
        coherent_amplitude_weights[:, np.newaxis, np.newaxis]
        * INTER_RX_PHASE_CORRECTIONS[:, np.newaxis, np.newaxis]
        * calibrated_images,
        axis=0,
    )
    coherent_power = np.abs(coherent_image) ** 2

    products = [
        (
            "baseline_equal_power",
            baseline_equal_power,
```

```
            baseline_images,
            baseline_coherence,
        ),
        (
            "calibrated_weighted_power",
            weighted_power,
            calibrated_images,
            calibrated_coherence,
        ),
        (
            "calibrated_coherent_experimental",
            coherent_power,
            calibrated_images,
            calibrated_coherence,
        ),
    ]
    for path_name, power, channel_images, coherence_maps in products:
        result, metric_row = summarize_image_power(
            run,
            path_name,
            power,
            channel_images,
            coherence_maps,
        )
        PIPELINE_RESULTS[run][path_name] = result
        metric_rows.append(metric_row)

    PIPELINE_RESULTS[run]["baseline_channel_images"] = baseline_images
    PIPELINE_RESULTS[run]["calibrated_channel_images"] = calibrated_images
    PIPELINE_RESULTS[run]["calibrated_channel_image_db"] = np.stack(
        [
            normalized_power_db(image, minimum_db=-60.0)
            for image in calibrated_images
        ]
    )

PIPELINE_METRICS = pd.DataFrame(metric_rows).set_index(
    ["processing_path", "run"]
)
for processing_path in PIPELINE_METRICS.index.get_level_values(0).unique():
    run_c_x = PIPELINE_METRICS.loc[
        (processing_path, "C"), "peak_cross_range_m"
    ]
    run_d_x = PIPELINE_METRICS.loc[
        (processing_path, "D"), "peak_cross_range_m"
    ]
    PIPELINE_METRICS.loc[
```

```
        (processing_path, "C"), "cross_range_shift_from_C_m"
    ] = 0.0
    PIPELINE_METRICS.loc[
        (processing_path, "D"), "cross_range_shift_from_C_m"
    ] = run_d_x - run_c_x

enhanced_shift_m = PIPELINE_METRICS.loc[
    ("calibrated_weighted_power", "D"),
    "cross_range_shift_from_C_m",
]
enhanced_status = "PASS" if enhanced_shift_m > 0 else "REVIEW"
print(
    f"Enhanced weighted-power Run D right-shift check: {enhanced_status} | "
    f"x_D - x_C = {enhanced_shift_m * 100:.2f} cm"
)
if enhanced_status != "PASS":
    warnings.warn(
        "The enhanced result did not preserve Run D's expected rightward shift.␣
 ↪"
        "Retain the baseline and review the transferred phase calibration."
    )

display(range_calibration_summary)
display(PHASE_CALIBRATION_SUMMARY)
display(RX_QUALITY_TABLE)
display(PIPELINE_METRICS)
```

```
Enhanced weighted-power Run D right-shift check: PASS | x_D - x_C = 3.70 cm
```

| | rx_channel | method | reference acquisition | \ |
|---|---|---|---|---|
| 0 | 0 | one-point additive-bias calibration | Run C | |
| 1 | 1 | one-point additive-bias calibration | Run C | |
| 2 | 2 | one-point additive-bias calibration | Run C | |
| 3 | 3 | one-point additive-bias calibration | Run C | |

| | laser-reference oversampled bin | equivalent original bin | \ |
|---|---|---|---|
| 0 | 175.295680 | 21.911960 | |
| 1 | 175.213904 | 21.901738 | |
| 2 | 175.104467 | 21.888058 | |
| 3 | 175.545082 | 21.943135 | |

| | nominal range at bin (m) | laser slant range (m) | range intercept b (m) | \ |
|---|---|---|---|---|
| 0 | 1.027123 | 0.84 | -0.187123 | |
| 1 | 1.026644 | 0.84 | -0.186644 | |
| 2 | 1.026003 | 0.84 | -0.186003 | |
| 3 | 1.028584 | 0.84 | -0.188584 | |

| | calibrated range at bin (m) |
|---|---|

```
0                        0.84
1                        0.84
2                        0.84
3                        0.84
```

```
   rx_channel  phase_rms_before_deg  phase_rms_after_deg  \
0           0             19.717115            15.654045
1           1             23.735061            11.822086
2           2             36.925775            12.920211
3           3             39.372432            16.578136

   maximum_applied_correction_deg
0                       59.299257
1                       61.757552
2                       65.206043
3                       65.686009
```

```
   rx_channel  target_to_reference_db  frame_coherence  \
0           0               32.337616         0.999997
1           1               32.151382         0.999996
2           2               33.509523         0.999997
3           3               36.702333         0.999997

   background_correlation  raw_quality_score  fusion_weight
0                0.992921          41.095464       0.208357
1                0.998352          40.443733       0.205052
2                0.999115          47.324947       0.239940
3                0.999455          68.372064       0.346651
```

```
                                    peak_cross_range_m  peak_downrange_m  \
processing_path                 run
baseline_equal_power            C               -0.008             0.838
calibrated_weighted_power       C               -0.008             0.838
calibrated_coherent_experimental C              -0.008             0.838
baseline_equal_power            D                0.029             0.850
calibrated_weighted_power       D                0.029             0.848
calibrated_coherent_experimental D               0.029             0.848

                                    peak_center_slant_range_m  \
processing_path                 run
baseline_equal_power            C                    0.838038
calibrated_weighted_power       C                    0.838038
calibrated_coherent_experimental C                   0.838038
baseline_equal_power            D                    0.850495
calibrated_weighted_power       D                    0.848496
calibrated_coherent_experimental D                   0.848496

                                    laser_slant_range_error_m  \
processing_path                 run
```

```
baseline_equal_power             C                   -0.001962
calibrated_weighted_power        C                   -0.001962
calibrated_coherent_experimental C                   -0.001962
baseline_equal_power             D                    0.010495
calibrated_weighted_power        D                    0.008496
calibrated_coherent_experimental D                    0.008496

                                      supporting_rx_coherence  \
processing_path                  run
baseline_equal_power             C                  0.963824
calibrated_weighted_power        C                  0.987004
calibrated_coherent_experimental C                  0.987004
baseline_equal_power             D                  0.934442
calibrated_weighted_power        D                  0.959628
calibrated_coherent_experimental D                  0.959628

                                      minus3db_cross_range_width_m  \
processing_path                  run
baseline_equal_power             C                      0.026675
calibrated_weighted_power        C                      0.025597
calibrated_coherent_experimental C                      0.025655
baseline_equal_power             D                      0.026739
calibrated_weighted_power        D                      0.025611
calibrated_coherent_experimental D                      0.025231

                                      minus3db_downrange_width_m  \
processing_path                  run
baseline_equal_power             C                     0.064017
calibrated_weighted_power        C                     0.064055
calibrated_coherent_experimental C                     0.063704
baseline_equal_power             D                     0.065336
calibrated_weighted_power        D                     0.065265
calibrated_coherent_experimental D                     0.065123

                                      peak_sidelobe_level_db  \
processing_path                  run
baseline_equal_power             C               -19.940113
calibrated_weighted_power        C               -19.927707
calibrated_coherent_experimental C               -26.873970
baseline_equal_power             D               -13.919904
calibrated_weighted_power        D               -14.756892
calibrated_coherent_experimental D               -25.378453

                                      integrated_sidelobe_ratio_db  \
processing_path                  run
baseline_equal_power             C                     -18.686727
calibrated_weighted_power        C                     -18.536982
calibrated_coherent_experimental C                     -27.472115
```

```
baseline_equal_power             D                      -14.383947
calibrated_weighted_power        D                      -14.433209
calibrated_coherent_experimental D                      -24.674672

                                      cross_range_shift_from_C_m
processing_path                  run
baseline_equal_power             C                           0.000
calibrated_weighted_power        C                           0.000
calibrated_coherent_experimental C                           0.000
baseline_equal_power             D                           0.037
calibrated_weighted_power        D                           0.037
calibrated_coherent_experimental D                           0.037
```

## 1.12 8. Final focused images

The comparison figure keeps the defined equal-power baseline beside the two enhanced outputs:

1. **Baseline equal-power fusion** — independently focus every RX and add their image powers with equal weights.
2. **Calibrated, quality-weighted power fusion** — apply the smooth Run C aperture-phase correction, then weight the RX powers using Run C quality measurements. This is the recommended pipeline output.
3. **Experimental coherent RX fusion** — phase-align the calibrated RX images using Run C and sum their complex fields. This branch is diagnostic until its transferability is confirmed on additional data.

Every panel is normalized independently to its own maximum and uses the same $-25$ dB display floor. The comparison therefore emphasizes localization, focus, and sidelobe structure rather than absolute received power. Cyan markers identify focused peaks. In every Run D panel, the dashed line marks the Run C peak cross-range for that same processing path.

The second figure displays the four phase-calibrated, independently focused RX images for each run. These panels document the inputs to the enhanced fusion stages and expose channel-specific artifacts. The final profile figure compares the three processing paths through each focused peak.

[13]:
```
image_extent_cm = [
    IMAGE_CROSS_RANGE_M[0] * 100,
    IMAGE_CROSS_RANGE_M[-1] * 100,
    IMAGE_DOWNRANGE_M[0] * 100,
    IMAGE_DOWNRANGE_M[-1] * 100,
]

processing_paths = (
    "baseline_equal_power",
    "calibrated_weighted_power",
    "calibrated_coherent_experimental",
)
processing_labels = {
    "baseline_equal_power": "Baseline equal power",
```

```
    "calibrated_weighted_power": "Calibrated weighted power",
    "calibrated_coherent_experimental": "Experimental coherent RX",
}

fig, axes = plt.subplots(
    2, 3, figsize=(16, 9), sharex=True, sharey=True, constrained_layout=True
)
image_handle = None
for row_index, run in enumerate(("C", "D")):
    for column_index, processing_path in enumerate(processing_paths):
        axis = axes[row_index, column_index]
        result = PIPELINE_RESULTS[run][processing_path]
        image_handle = axis.imshow(
            result["power_db"],
            origin="lower",
            extent=image_extent_cm,
            aspect="equal",
            cmap="viridis",
            vmin=IMAGE_DISPLAY_FLOOR_DB,
            vmax=0.0,
        )
        axis.plot(
            result["peak_cross_range_m"] * 100,
            result["peak_downrange_m"] * 100,
            marker="x",
            color="cyan",
            markersize=9,
            markeredgewidth=2,
            label="Focused peak",
        )
        if run == "D":
            run_c_peak_x_m = PIPELINE_RESULTS["C"][processing_path][
                "peak_cross_range_m"
            ]
            axis.axvline(
                run_c_peak_x_m * 100,
                color="white",
                linestyle="--",
                linewidth=1.2,
                label="Run C peak cross-range",
            )
        axis.set_title(f"Run {run} - {processing_labels[processing_path]}")
        axis.set_xlabel("Cross-range, $x$ (cm)")
        axis.set_ylabel("Downrange, $z$ (cm)")
        axis.legend(loc="upper right", fontsize=8)

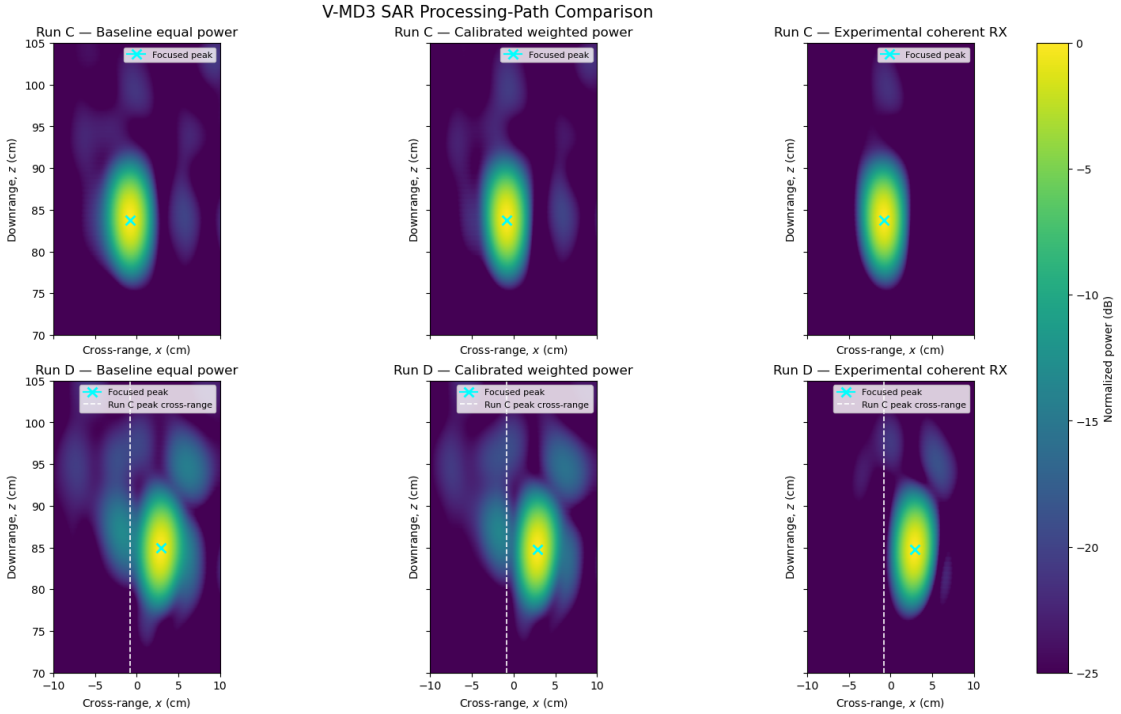
fig.colorbar(image_handle, ax=axes, label="Normalized power (dB)")
```

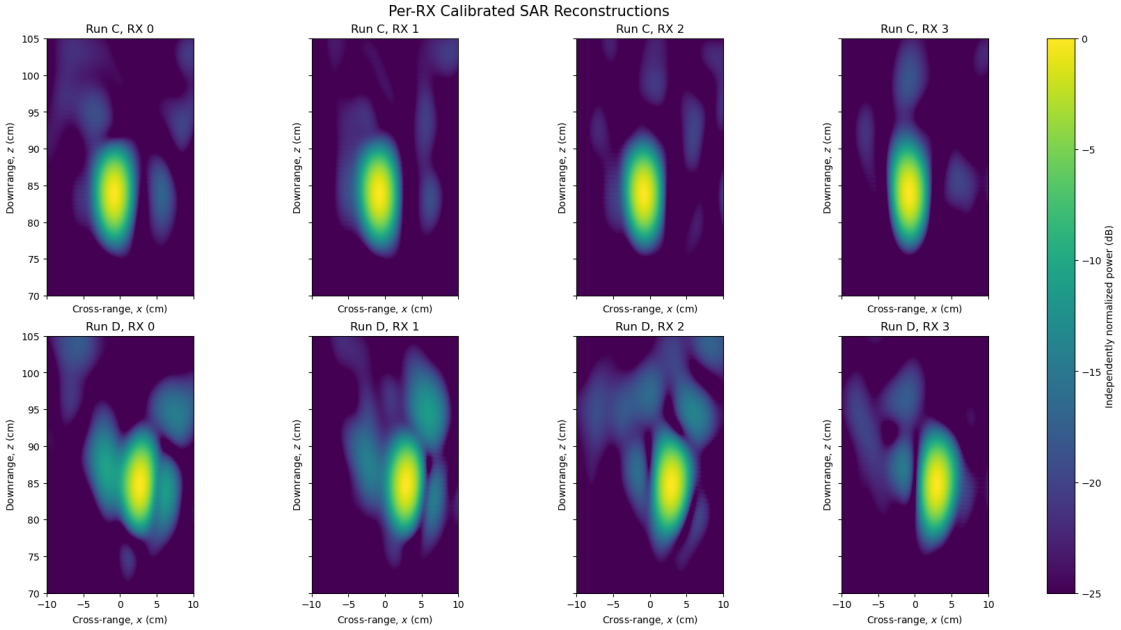
```
fig.suptitle("V-MD3 SAR Processing-Path Comparison", fontsize=15)
plt.show()

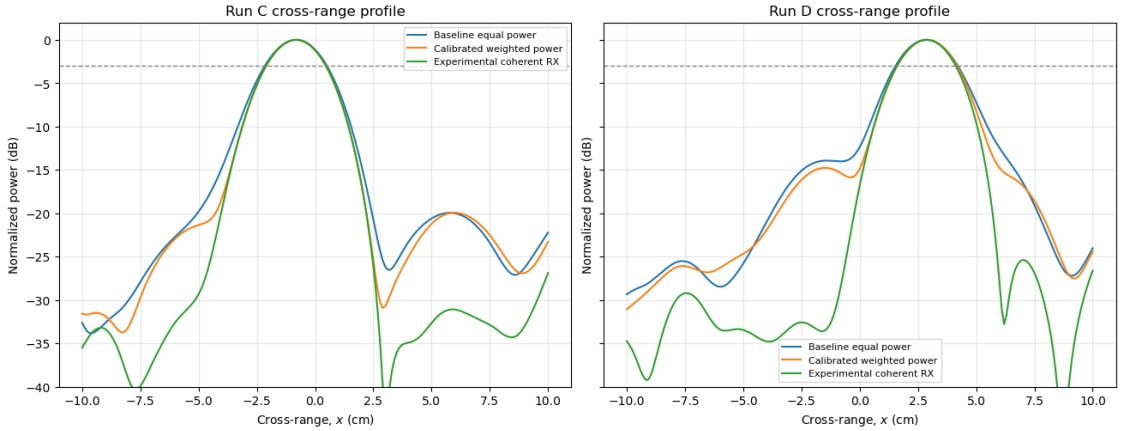
fig, axes = plt.subplots(
    2, 4, figsize=(17, 9), sharex=True, sharey=True, constrained_layout=True
)
channel_handle = None
for row_index, run in enumerate(("C", "D")):
    for rx_channel in RX_CHANNELS:
        axis = axes[row_index, rx_channel]
        channel_handle = axis.imshow(
            PIPELINE_RESULTS[run]["calibrated_channel_image_db"][rx_channel],
            origin="lower",
            extent=image_extent_cm,
            aspect="equal",
            cmap="viridis",
            vmin=IMAGE_DISPLAY_FLOOR_DB,
            vmax=0.0,
        )
        axis.set_title(f"Run {run}, RX {rx_channel}")
        axis.set_xlabel("Cross-range, $x$ (cm)")
        axis.set_ylabel("Downrange, $z$ (cm)")
fig.colorbar(
    channel_handle,
    ax=axes.ravel().tolist(),
    label="Independently normalized power (dB)",
)
fig.suptitle("Per-RX Calibrated SAR Reconstructions", fontsize=15)
plt.show()

fig, axes = plt.subplots(
    1, 2, figsize=(13, 5), sharey=True, constrained_layout=True
)
for axis, run in zip(axes, ("C", "D")):
    for processing_path in processing_paths:
        result = PIPELINE_RESULTS[run][processing_path]
        axis.plot(
            result["cross_range_profile_coordinate_m"] * 100,
            result["cross_range_profile_db"],
            linewidth=1.5,
            label=processing_labels[processing_path],
        )
    axis.axhline(-3.0, color="gray", linestyle="--", linewidth=1.0)
    axis.set_title(f"Run {run} cross-range profile")
    axis.set_xlabel("Cross-range, $x$ (cm)")
    axis.set_ylabel("Normalized power (dB)")
    axis.set_ylim(-40, 2)
```

```
    axis.grid(True, alpha=0.3)
    axis.legend(fontsize=8)
plt.show()
```






## 1.13 9. Interpretation and reproducibility checks

The defined equal-power result is retained as the reference baseline. The recommended enhanced output is the **calibrated, quality-weighted power fusion** image. Adopt an enhancement only when its metrics and image structure improve without changing the known physical interpretation: Run C remains near the center, Run D moves to the right, and both targets remain near their independently measured 0.84 m slant range.

RX 3 remains available as a regression check against the source notebook. Its independently focused Run C image should remain close to the previously selected result: approximately $x = -0.6$ cm, $z = 83.4$ cm, and peak focusing coherence $\approx 0.975$. The fused peak and measured widths can differ because all four channels contribute.

Run D is processed with the same four RX channels, windows, FFT size, phase sign, background-reference rule, fractional range-shift search, Run C-derived range calibration, Run C-derived aperture-phase correction, fixed RX quality weights, and image grid. Its matched-background coefficient and small range-grid shift are estimated in the normal per-run subtraction step, but no range, aperture-phase, or fusion calibration is refit on Run D. Its principal validation criterion is

$$x_{\mathrm{peak,D}} - x_{\mathrm{peak,C}} > 0,$$

meaning that the offset target focuses to the right under the displayed positive-$x$ convention. If the check fails, inspect the physical rail-coordinate direction and target-placement record before changing the processing parameters.

Both targets were independently laser-measured at a slant range of 0.84 m. The metrics table reports focused center-aperture slant range, range error, peak coherence, $-3$ dB widths, peak sidelobe level, integrated sidelobe ratio, and Run D's cross-range displacement relative to Run C. Because Run C defines the one-point range correction, its absolute range agreement is expected; Run D tests whether that correction transfers. More datasets at different measured ranges are needed to determine whether the sensor also has a range-scale error.

The coherent RX branch is intentionally labeled **experimental**. It uses Run C to estimate a constant phase relationship between receiver images, then transfers that relationship unchanged to

Run D. It should replace neither the baseline nor the weighted-power output until additional target positions and repeat acquisitions show that its apparent improvement is stable.

### 1.13.1 Current limitations

- Fractional-bin interpolation improves the estimate of the Run C reference peak, but the range calibration remains a one-point additive-bias correction. A future dedicated calibration dataset with multiple known slant ranges should determine whether a range-scale error is also present.
- Fractional background alignment compensates small range-grid offsets between the target and background captures. Large shifts or solutions pinned at the search boundary indicate an acquisition mismatch and should be reviewed rather than accepted automatically.
- The smooth aperture-phase correction is empirical and Run C-derived. Its low polynomial order limits overfitting, but transfer to more scenes is still required before treating it as a permanent instrument calibration.
- Quality weights are frozen from Run C and capped so that one receiver cannot dominate the result. They improve robustness only if the measured Run C channel quality represents later acquisitions.
- Each RX is treated with a monostatic phase-center approximation because the exact V-MD3 TX geometry is not yet independently measured.
- The primary four-RX output is fused in power. The coherent comparison remains provisional because a fully transferable coherent combination requires independently characterized TX/RX geometry and inter-channel phase calibration.
- Existing cross-range grating lobes arise primarily because the 6.25 mm rail-position step is greater than half a wavelength at 61 GHz. Post-processing can change their appearance but cannot recover spatial samples that were not acquired; a wider unambiguous field of view ultimately requires denser rail sampling.